

Vietnam General Confederation of Labor

Ton Duc Thang University

Faculty of Information Technology



Bui Phuong Nam – 522H0002

Hoang Minh Quan – 52300245

A Comparison of Monolithic and Microservices Architectures in LMS Systems

**INFORMATION TECHNOLOGY PROJECTS
SOFTWARE ENGINEERING**

Supervisor

Msc. Mai Van Manh

Ho Chi Minh City, 2026

Acknowledgment

First and foremost, I would like to express my sincere gratitude to my supervisors, **MSc. Mai Van Manh**, for his invaluable guidance, constructive feedback, and continuous encouragement throughout the duration of this thesis. His expertise, patience, and dedication have played a crucial role in shaping both the technical direction and academic rigor of this research.

I am also deeply thankful to the lecturers and staff of the Faculty of Information Technology at Ton Duc Thang University, whose support and resources provided a strong foundation for my research journey. Their commitment to creating an inspiring and intellectually stimulating environment has greatly contributed to my growth.

My gratitude further extends to my colleagues, peers, and friends who offered helpful discussions, motivation, and meaningful insights during the development and evaluation of this project. Their companionship made the challenging moments far more manageable.

Finally, I am profoundly grateful to my family for their unconditional support, patience, and encouragement. Their belief in me has always been my greatest source of strength and determination.

To all of you, I extend my heartfelt thanks.

Certification

This Projects was carried out at **Ton Duc Thang University**.

Advisor: MSc. Mai Van Manh

.....
(Title, Full Name and Signature)

This project was defended at the Undergraduate Thesis Examination Committee held at Ton Duc Thang University on / /

Confirmation of the Chairman of the Undergraduate Thesis Examination Committee and the Dean of the Faculty after receiving the revised thesis (if any).

CHAIRMAN

DEAN OF FACULTY

.....

Declaration

This project was completed at **Ton Duc Thang University**. We hereby declare that this is our own research work, conducted under the academic supervision of **MSc. Mai Van Manh**. All analyses, discussions, and results presented in this project are truthful, original, and have not been published in any form prior to this submission.

All data used for analysis, evaluation, and comparison were collected directly by the author(s) from various reliable sources, which are fully cited in the reference section of this project. In addition, certain evaluations, comments, and figures quoted from other authors, organizations, or research works are clearly referenced and properly acknowledged.

If any form of academic dishonesty, plagiarism, or copyright infringement is discovered, we accept full responsibility for the content of this project. Ton Duc Thang University shall bear no liability for any copyright or authorship violations committed by the author(s) during the execution of this research.

Ho Chi Minh City, day month year

Author(s)

(Signature and full name)

Chapter 1

Abstract

The selection of an appropriate software architecture plays a critical role in the development and evolution of **Learning Management Systems (LMS)**, directly affecting system scalability, maintainability, and adaptability to changing requirements. This project presents a comparative study of three widely adopted architectural approaches—**Monolithic**, and **Microservices**—within the context of an LMS application.

A core LMS system was designed with essential functionalities, including user management, course management, enrollment, learning content delivery, and assessment handling. Based on the same functional requirements, the system was implemented separately using the three architectural styles to ensure a fair and consistent comparison. Several change and extension scenarios, such as modifying grading logic, adding learning progress statistics, and preparing for future learning analytics integration, were defined and applied to each architecture.

The architectures were evaluated using technical criteria including module isolation, change impact scope, scalability at the functional level, deployment and operational complexity, and suitability for different system scales. The experimental results highlight the strengths and limitations of each approach: Monolithic architecture offers simplicity and ease of deployment but exhibits limited flexibility when evolving; Monolithic improves maintainability through clearer module boundaries while retaining centralized deployment; Microservices provide superior scalability and change isolation at the cost of increased implementation and operational complexity.

This study provides practical insights into the real-world trade-offs of architectural choices and proposes recommendations for selecting suitable architectures for LMS systems of small, medium, and large scales.

Contents

Acknowledgment	i
Certification	ii
Declaration	iii
1 Abstract	iv
Abstract	iv
Contents	v
2 Introduction	1
2.1 Background and Rationale	1
2.2 Project Objectives	1
2.3 Scope of the Project	3
2.4 Research Methodology	4
2.4.1 Theoretical Framework and Comparison Basis	4
2.4.2 Experimental Design and Implementation	4
2.4.3 Quantitative Evaluation Metrics	4
2.4.4 Maintainability and Evolution Scenarios	5
3 Literature Review	6
3.1 Related Works	6
3.1.1 Architectural Evolution in E-Learning Systems	6
3.1.2 The CAP Theorem and Architectural Trade-offs	7
3.1.3 Empirical Performance Comparisons	7
3.1.4 Architectural Patterns	8
3.2 Theoretical Principles and Architectural Models	9
3.2.1 Monolithic Architecture	9
3.2.2 Microservices Architecture	10
3.3 Comparison Methodology	10
3.3.1 Throughput & Response time	11
3.3.2 CPU Usage	15
3.3.3 Fault Tolerance	18
3.3.4 Cost Efficiency	19

3.4	Relevant core Technologies	20
3.4.1	NestJS Framework	20
3.4.2	PostgreSQL	21
3.4.3	Communication Paradigms: Synchronous vs. Asynchronous	21
3.4.4	Docker and Containerization	22
4	System Design	23
4.1	System stakeholders analysis	23
4.2	Detailed Backend Architectures	24
4.2.1	Monolithic organization	24
4.2.2	Microservices organization	26
4.2.3	Code Conventions and Naming Standards	28
4.2.4	Architectural implementation Summary	28
4.3	Architectural Decomposition Strategy	31
4.3.1	Monolithic Structural Cohesion	31
4.3.2	Microservices Granularity and Isolation	32
4.4	Communication and Integration Patterns	33
4.5	Infrastructure and Deployment Topology	34
5	Experimental Setup	35
5.1	Hardware and Software Environment	35
5.2	System-Under-Test: Architecture Configurations	36
5.3	Virtual User Interaction Model	36
5.4	Load Scenarios	37
5.5	Measurement quantities	37
5.5.1	Throughput (RPS — Requests Per Second)	38
5.5.2	Response Time — Average, P95, and P99	38
5.5.3	CPU Usage and host_total	39
5.5.4	RAM Usage	39
5.6	Metric Definitions	40
5.6.1	Resource Usage Efficiency (RUE)	40
5.6.2	Availability, MTBF, and MTTR	41
5.7	Benchmark Design Rationale	44
6	Experimental results and analysis	45
6.1	Analysis of Throughput & Response Time	45
6.1.1	Scenario 1: Gradual Ramp-Up Load	45
6.1.2	Scenario 2: Peak Load (Flash Crowd Simulation)	47
6.2	Analysis of CPU Usage	49
6.2.1	Scenario 1: Read-Only Access (GET /courses)	49
6.2.2	Scenario 2: Write-Intensive Access (POST /profile)	51
6.2.3	Scenario 3: Mixed Concurrent Load	52
6.3	Fault Tolerance Analysis	54
6.4	Cost Efficiency Analysis	56

7	Discussion	60
7.1	The Performance-Scalability Duality	60
7.1.1	Communication Friction at Low Load	60
7.1.2	Concurrency Saturation and the Architectural Crossover	61
7.1.3	Flash Crowd Behavior and Parallelism Advantage	61
7.1.4	The Resource Efficiency Paradox	62
7.1.5	The Infrastructure Tax and Its Load-Dependent Nature	62
7.2	Peak Load Efficiency Inversion	62
7.2.1	Reconciling the Efficiency Paradox	63
7.2.2	The Resilience-Complexity Trade-off	63
7.2.3	Fault Isolation as Architectural Resilience	63
7.2.4	The MTTR Complexity Penalty	64
7.2.5	Architectural Suitability for LMS Deployments	65
7.2.6	The Enrollment Period Challenge	65
7.2.7	Scale-Dependent Architecture Recommendations	65
7.2.8	Limitations and Boundary Conditions	66
8	Conclusion	67
8.1	Overview of the Study	67
8.2	Key Findings and Insights	67
8.2.1	Throughput and Response Time: The Concurrency Crossover	67
8.2.2	CPU Utilization and Resource Efficiency: The Infrastructure Tax	68
8.2.3	Fault Tolerance and Development-Phase Resilience	68
8.3	Contextual Superiority	68
8.4	Final Remarks	69
	Bibliography	70

List of Figures

2.1	Experimental Comparison Framework: Monolithic vs. Microservices	5
3.1	Monolithic concept	10
3.2	Microservice concept.	10
3.3	Throughput comparison under graduated load testing.	12
3.4	Response time progression under graduated load testing.	12
3.5	number of request under graduated load testing	13
3.6	Throughput under simultaneous request load.	13
3.7	Response time under concurrent load conditions.	14
3.8	Nestjs overview	21
3.9	Postgre database associated with nestjs	21
4.1	Repository pattern design for Monolithic architect	25
4.2	Microservices Granularity and Isolation	33
5.1	Throughput concept.	38
5.2	CPU usage concept.	39
5.3	Ram usage concept.	40
5.4	Example of Log commit history by summary table	41
6.1	Throughput Comparison under Gradual Ramp-Up Load (Scenario 1)	46
6.2	Response Time Comparison under Gradual Ramp-Up Load (Scenario 1)	47
6.3	Throughput under Simultaneous Peak Load (Scenario 2)	48
6.4	Response Time under Simultaneous Peak Load (Scenario 2)	48
6.5	Average CPU Usage across Load Levels — TC1 Read-Only (Monolith vs. Microservices)	50
6.6	CPU Usage Timeline — Monolith, 1 VU (Read-Only)	50
6.7	CPU Usage Timeline — Microservices, 1 VU (Read-Only)	51
6.8	Average CPU Usage across Load Levels — TC2 Write-Intensive (Monolith vs. Microservices)	51
6.9	Average CPU Usage across Load Levels — TC3 Mixed Concurrent Load (Monolith vs. Microservices)	52
6.10	CPU Usage Timeline — Monolith, 2,000 VUs (Mixed Load)	53
6.11	CPU Usage Timeline — Microservices, 2,000 VUs (Mixed Load)	53
6.12	Result experience statistic in microservice	54
6.13	Result experience statistic in monolithic	55
6.14	Resource consumption comparison under Normal Load (200 VUs)	57
6.15	Resource consumption comparison under Peak Load (1000 VUs)	59

List of Tables

3.1	Comparison of related works across evaluation dimensions (✓ = covered, Partial = limited coverage, ✗ = not covered)	8
4.1	Entity Storage Description	29
4.2	Service Storage Description	30
5.1	System configuration for benchmarking environment	35
5.2	Virtual User Interaction Cycle (4-step sequence, 1 s think time between cycles)	37
5.3	Load Scenario Definitions	37
6.1	Baseline Resource Consumption under Idle Load	57
6.2	Aggregated performance metrics and resource consumption (Normal Load - 200 VUs)	58
6.3	Aggregated performance metrics and resource consumption (Peak Load - 1000 VUs)	58

Chapter 2

Introduction

2.1 Background and Rationale

The global education landscape has undergone a profound paradigm shift, accelerated by the increasing demand for remote learning and digital literacy. Learning Management Systems (LMS) have transitioned from being supplementary tools to becoming the core infrastructure of modern academic institutions. As user bases grow from hundreds to millions of concurrent learners, the underlying software architecture of these platforms faces unprecedented pressure. The architectural integrity of an LMS is no longer just a technical detail; it is a strategic foundation that determines the system's ability to remain available under seasonal traffic spikes, such as during national examinations or enrollment periods.

Traditionally, software development favored the *Monolithic* approach due to its simplicity in initial deployment and cohesive data management. However, as systems evolve in complexity, the rigid coupling inherent in Monoliths often leads to "architectural erosion," where minor changes in one module can trigger unpredictable failures across the entire system. In response, the *Microservices* architecture has emerged as a disruptive alternative, promising independent scalability and fault isolation by decomposing the system into smaller, loosely coupled services.

Despite the widespread adoption of Microservices by industry giants, it introduces a "complexity tax," involving distributed networking overhead and data consistency challenges. This research is motivated by the need to provide a rigorous, empirical comparison between these dominant paradigms within the specific functional requirements of an LMS—covering identity management, course delivery, and high-stakes assessment. By quantifying the trade-offs in performance, resilience, and resource efficiency, this study aims to provide a decision-making blueprint for educational technology architects.

2.2 Project Objectives

The overarching goal of this research is to architect, implement, and rigorously evaluate a Learning Management System (LMS) across contrasting architectural paradigms. The study aims to move beyond theoretical comparisons by providing an empirical data-driven analysis of how Monolithic and Microservices models behave under identical operational pressures, ultimately determining their suitability for various organizational scales and technical requirements.

To achieve this, the project is structured around five core research objectives:

1. **High-Fidelity System Design and Domain Specification:** The first objective focuses on designing a comprehensive LMS framework that serves as a stable "functional baseline." This involves:
 - Developing a detailed Software Requirements Specification (SRS) for core academic workflows, including Identity and Access Management (IAM), Course Lifecycle Management, Content Delivery, and complex Assessment engines.
 - Utilizing Domain-Driven Design (DDD) to identify "Bounded Contexts," ensuring that the business logic is modular enough to be implemented as either internal packages (Monolith) or independent services (Microservices) without altering the core functionality.
2. **Dual-Architectural Implementation and Environment Parity:** The project aims to develop two distinct, production-ready versions of the LMS. The focus is to maintain a *ceteris paribus* (all other things being equal) condition:
 - Implementing the Monolithic version as a single-process application using NestJS, ensuring centralized data management and low-latency internal communication.
 - Implementing the Microservices version using a "Database-per-Service" pattern, coordinated via an API Gateway and asynchronous message brokering (RabbitMQ).
 - Utilizing Docker and Docker Compose to ensure both architectures operate within identical infrastructure constraints, eliminating environment-induced performance variances.
3. **Empirical Performance Benchmarking and Stress Testing:** To quantify the "Performance Tax" of each architecture, the study will conduct systematic load tests:
 - Measuring raw Throughput (Requests Per Second) and Tail Latency (P95/P99) under diverse load profiles, including "Step Loads" for scalability and "Flash Crowds" for peak stress resilience.
 - Calculating the Resource Utilization Efficiency (RUE) to determine the marginal CPU and RAM consumption per active learner, providing a clear picture of the infrastructure costs associated with each model.
4. **Simulation of Software Evolution and Maintainability:** This objective evaluates how each architecture responds to the inevitable changes in a software lifecycle. The study will model "Controlled Change Scenarios" to:
 - Measure the development effort and "Ripple Effect" (regression risks) when modifying core business logic, such as grading algorithms.
 - Analyze the complexity of "Schema Evolution" and feature extensions (e.g., adding a learning analytics module) to observe the trade-offs between shared and isolated data stores.
5. **Comparative Synthesis and Decision-Support Framework:** The final objective is to synthesize all quantitative and qualitative data into a coherent evaluation matrix. This involves:
 - Analyzing the balance between "System Complexity" and "Performance Gains" to identify the "Break-even Point" where the overhead of Microservices becomes a justified investment.
 - Providing a strategic recommendation model for educational institutions, helping them choose an architecture based on their specific constraints in terms of team size, traffic variability, and long-term maintainability.

2.3 Scope of the Project

The primary focus of this project is a comparative analysis of software architectural paradigms; the LMS platform serves as a standardized experimental vehicle rather than a commercial end-product. To ensure the scientific validity of the comparison, the scope is strictly defined within the following boundaries:

- **Functional Stability and Code Parity:** Across all architectural implementations, the core business logic—encompassing Identity Management, Enrollment Workflows, Course Content Delivery, and Assessment Engines—remains functionally identical. A "Code Parity" policy is enforced: any algorithmic optimization made in the Monolithic version is mirrored in the Microservices version. This ensures that observed performance variances are strictly a result of architectural topology (e.g., network hops, serialization) rather than disparities in code efficiency.
- **Data Architecture and Persistence Scope:** The study covers two distinct data management strategies. For the Monolithic model, the scope includes a centralized relational schema within a single PostgreSQL instance. For the Microservices model, the scope shifts to a "Database-per-Service" pattern, utilizing isolated schemas to evaluate the impact of data decentralization on system consistency and complexity. The research limits its focus to relational data; NoSQL or Graph databases are excluded to maintain a controlled comparison environment.
- **Technological and Infrastructure Constraints:** The research utilizes a unified, industry-standard technology stack to eliminate "language-induced" bias.
 - **Backend:** NestJS (Node.js) is chosen for its modular architecture, allowing for a seamless transition between monolithic modules and microservices.
 - **Communication:** The scope includes synchronous RESTful APIs for the Monolith and a hybrid of REST and asynchronous Message Brokering (via RabbitMQ) for Microservices.
 - **Deployment:** Both systems are containerized using *Docker* and orchestrated via *Docker Compose*. The evaluation is conducted within a controlled virtual network to ensure "Environmental Parity," where CPU and RAM limits are strictly enforced at the container level.
- **Evaluation Metrics and Analytical Boundaries:** The project limits its quantitative analysis to specific Key Performance Indicators (KPIs): Throughput (Requests Per Second), Tail Latency (P95/P99), and Resource Utilization Efficiency (RUE). Qualitative analysis is bounded by "Controlled Change Scenarios" designed to simulate real-world maintenance tasks.
- **Exclusions and Out-of-Scope Elements:** To maintain a focused research trajectory, the following elements are explicitly excluded:
 - **Front-end Complexity:** The User Interface (UI) is developed as a "Minimum Viable Product" (MVP) using Vue.js, prioritizing functionality for data verification over aesthetic design or advanced User Experience (UX) features.
 - **Third-party Integrations:** External services such as Auth0, payment gateways, or complex Learning Record Stores (LRS) are excluded to avoid "Noise" from external network latencies.
 - **Advanced Architectures:** Alternative models such as Serverless, fully decentralized Event-Driven Architectures (EDA), or Service Meshes (e.g., Istio) are reserved for future work and are outside the current comparative framework.

- **Security Auditing:** While standard JWT-based authentication is implemented, deep penetration testing and exhaustive security audits are not within the scope of this architectural study.

2.4 Research Methodology

This study adopts a comparative and empirical research design, emphasizing a quantitative and qualitative architectural analysis. The methodology is structured to isolate architectural variables, ensuring that performance and operational differences observed between the *Monolithic* and *Microservices* implementations are attributable solely to their structural paradigms.

2.4.1 Theoretical Framework and Comparison Basis

The foundational phase involves a systematic review of architectural design patterns. By synthesizing contemporary theories [1], a standardized evaluation framework is established, focusing on three analytical pillars:

- **Performance and Scalability Efficiency:** Evaluating throughput and response-time degradation curves under elastic workloads.
- **Fault Tolerance and Resilience Topology:** Investigating the "Blast Radius" of localized failures and the system's ability to maintain partial availability.
- **Resource Optimization:** Measuring the marginal resource consumption (CPU/RAM) required per unit of throughput in a cloud-native context.

2.4.2 Experimental Design and Implementation

The core research involves implementing the LMS twice, following different architectural blueprints but maintaining strict business logic parity.

Functional Domain Decomposition

Using a Software Requirements Specification (SRS), the LMS is decomposed into *Bounded Contexts*. This ensures that domains like *Assessment* and *Course Management* are consistent across versions, eliminating software-level bias and maintaining a *ceteris paribus* condition.

Environment Control

Both architectures are orchestrated via *Docker Compose*. This approach guarantees environment parity, ensuring that networking latency and runtime overhead are consistent across all test cycles.

2.4.3 Quantitative Evaluation Metrics

The study employs scenario-based stress testing to extract granular data:

1. **Performance (Throughput & Latency):** Measuring Requests Per Second (RPS) and P95/P99 "Tail Latency" to identify system bottlenecks.
2. **Resource Utilization Efficiency (RUE):** Calculating the Resource Utilization Index to measure the CPU/RAM processing capacity consumed relative to workload intensity.

3. **Fault Tolerance Assessment:** Measuring *Mean Time To Recovery* (MTTR) and the *Service Availability Index* after injecting failures into non-critical modules.
4. **Total Cost of Ownership (TCO):** Analyzing infrastructure footprint and operational complexity involved in deployment and service orchestration.

2.4.4 Maintainability and Evolution Scenarios

Three "Change Scenarios" are introduced to simulate long-term maintenance:

- **Logic Modification:** Assessing regression risks during algorithm updates.
- **Feature Extension:** Measuring the effort to integrate learning analytics.
- **Schema Evolution:** Evaluating the impact of data structure changes on interconnected modules (Shared DB vs. Isolated DB).

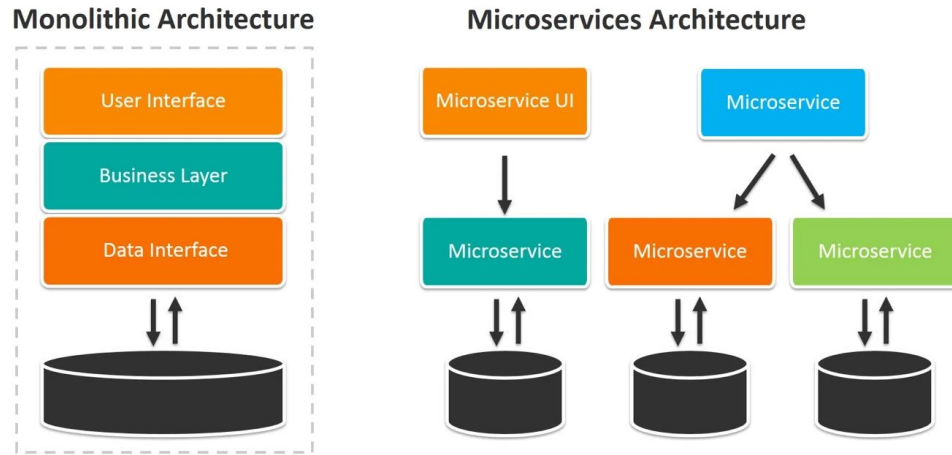


Figure 2.1: Experimental Comparison Framework: Monolithic vs. Microservices

Chapter 3

Literature Review

This chapter provides a comprehensive synthesis of the theoretical foundations and empirical evidence governing the transition from Monolithic to Microservices architectures, specifically within the domain of Learning Management Systems (LMS). By critically evaluating established principles such as the CAP theorem, the "Infrastructure Tax" phenomenon, and modularity patterns, this review identifies a significant research gap: the lack of multi-dimensional, LMS-specific benchmarks that integrate performance, resource efficiency, and development-phase resilience. The following sections establish the conceptual vocabulary and methodological precedents—ranging from host-level CPU monitoring to availability metrics—that form the baseline for this study's comparative framework. Ultimately, this review moves beyond the binary "Monolith versus Microservices" debate to explore how architectural topology dictates system behavior under the heterogeneous read/write workloads characteristic of modern educational environments.

3.1 Related Works

This section reviews the existing body of knowledge regarding architectural evolution, performance trade-offs, and empirical comparisons between Monolithic and Microservices paradigms. By synthesizing findings from recent studies and industry reports, this review establishes the theoretical framework and identifies the research gaps that this project aims to address through its comparative LMS implementation.

3.1.1 Architectural Evolution in E-Learning Systems

Learning Management Systems have undergone substantial architectural evolution over the past two decades. Early platforms such as Moodle, Blackboard, and Canvas were designed as tightly-coupled monolithic applications to simplify deployment in institutional settings where a single server and unified codebase minimised operational overhead [2]. Their architecture reflected the prevailing assumption that a university's user population would remain bounded and predictable — an assumption invalidated by the rapid growth of Massive Open Online Courses (MOOCs) and, more recently, by the COVID-19 pandemic's surge in remote education, which temporarily multiplied concurrent user loads by factors of three to five at many institutions [3].

As LMS deployments scaled to tens of thousands of concurrent learners, the structural limitations of shared-state, single-process architectures became apparent in at least three ways. First, database connection pools shared across all modules (authentication, course content, quiz grading) created contention hotspots: a spike in quiz submissions degraded course-browsing response times for unrelated users. Second, scaling the

entire monolith to address a bottleneck in one module (e.g., video streaming) was operationally wasteful and financially costly, as it replicated underutilised capacity. Third, the tight coupling of modules within a shared codebase meant that even minor changes — such as updating a grading rubric — required full system rebuilds and re-deployments, extending Mean Time To Recovery (MTTR) and reducing development velocity.

Niemelä and Hyyrö (2019) were among the first to formally propose a migration pathway from a monolithic LMS to a microservices architecture, arguing that service decomposition along pedagogical boundaries (content delivery, assessment, analytics) aligns naturally with the modular structure of course design and enables independent scaling of high-demand subsystems [2]. Their framework, however, remained theoretical and lacked empirical performance data. Dinh (2023) subsequently documented the technical debt accumulation patterns in a production monolithic LMS deployed at a Vietnamese university, reporting that over 60% of system downtime incidents were attributable to database lock contention caused by the absence of service-level isolation, and proposing a phased decomposition strategy [4]. Hasan et al. (2023) extended this line of work by measuring the maintainability impact of architectural migration using code-level metrics (cyclomatic complexity, afferent and efferent coupling), finding that a microservices refactoring of a higher-education information system reduced cross-module dependencies by up to 43% and cut the average number of files touched per feature change by approximately 60% [3].

3.1.2 The CAP Theorem and Architectural Trade-offs

In the context of distributed E-learning systems, the CAP theorem (Consistency, Availability, and Partition Tolerance) serves as a foundational theoretical constraint. A Monolithic LMS inherently prioritizes **Consistency (C)** through its unified database and ACID transactions, ensuring that a student's grade or enrollment status is immediately reflected system-wide. However, this often comes at the expense of **Availability (A)** during high-traffic surges, where a single point of failure can paralyze the entire application.

Conversely, the transition to Microservices necessitates a shift toward **Availability** and **Partition Tolerance (P)**. By adopting an "Eventual Consistency" model, Microservices allow individual modules (e.g., Course Content) to remain accessible even if the Grading service is under heavy load or temporarily unavailable. This research examines how this theoretical shift impacts the user experience during peak academic periods, balancing the need for immediate data accuracy against the necessity of continuous system uptime.

3.1.3 Empirical Performance Comparisons

A substantial body of empirical literature has attempted quantitative performance comparisons between Monolithic and Microservices architectures, though the results are highly context-dependent. Al-Debagy and Martinek (2018) performed controlled load tests on functionally equivalent RESTful applications, reporting that throughput differences between the two architectures averaged only 0.87% under gradually increasing load, but diverged to approximately 6% under simultaneous concurrent request injection, with the Monolith retaining a throughput advantage at that specific scenario because the absence of inter-service network hops allows it to process requests with lower per-request overhead [5]. Their study established the methodological baseline adopted by subsequent researchers: using the same business-domain endpoints, identical hardware, and two distinct load scenarios (gradual ramp-up and concurrent surge).

Schlögl et al. (2024) refined this methodology by decomposing resource consumption measurement into three distinct test cases based on operation complexity: simple database query (analogous to GET /courses), intensive computation (data transformation), and distributed multi-service orchestration (analogous to enrolment workflows spanning authentication, course validation, and progress tracking) [6]. Their key contribution

was the observation that measuring CPU only at the application-service layer systematically underestimates Microservices overhead: only by aggregating host-level container metrics (equivalent to `host_total` in the current study) does the full cost of API Gateway routing, service discovery, and health-check polling become visible. This methodological insight directly informed the measurement design of the present study, which uses `host_total` as the primary CPU comparison metric for Microservices.

Tapia et al. (2020) conducted a systematic review of 31 primary studies comparing monolithic and microservices performance, finding that reported results were highly sensitive to the application domain, hardware environment, and load pattern used [Tapia et al., 2020]. Particularly relevant to the present work is their finding that e-commerce and financial systems — the domains most commonly studied — exhibit workload patterns (uniform transaction types, moderate concurrency) that differ substantially from the uneven read/write distributions characteristic of LMS deployments. This contextual gap is one of the primary motivations for the present study. Faye and Sheikh (2025) complemented these performance findings with a cost analysis across 23 architectural migration cases, reporting an average infrastructure cost increase of 47% and a DevOps tooling cost increase of 150–275% when transitioning to Microservices — findings that contextualise the Resource Usage Efficiency (RUE) metric adopted in this project’s cost analysis [7].

3.1.4 Architectural Patterns

Recent literature has begun questioning the binary framing of ‘Monolith versus Microservices’, proposing instead a spectrum of architectural options. Dragoni et al. (2017) provided a comprehensive theoretical analysis of the microservices paradigm, identifying accumulated serialisation/deserialisation costs and connection pool management as the primary mechanisms through which inter-service latency accumulates under high concurrency [8]. Taibi et al. (2018) mapped the most commonly adopted microservices architectural patterns across 60 industrial case studies, finding that the API Gateway (adopted in 87% of cases) and the Service Mesh pattern (adopted in 41%) are the dominant approaches to managing inter-service communication overhead [Taibi et al., 2018]. Xu (2025) synthesised these findings into a practitioner-oriented decision framework, concluding that for teams with fewer than 10 developers or systems with fewer than 1,000 daily active users, the operational overhead of microservices rarely delivers proportional resilience or scalability benefits — a finding particularly relevant to small and mid-sized LMS deployments [9].

Summary Comparison of Prior Studies and Research Gap

The table below synthesises the most relevant prior studies along four evaluation dimensions:

Table 3.1: Comparison of related works across evaluation dimensions (✓ = covered, Partial = limited coverage, ✗ = not covered)

Study	Domain	Performance	CPU / Resource	Fault Tolerance
Al-Debagy & Martinek (2018)	Generic REST	✓	✗	✗
Schlögl et al. (2024)	E-commerce	Partial	✓	✗
Parvatha (2020)	General theory	✗	✗	✓
Hasan et al. (2023)	HE Information Sys.	✗	✗	Partial
Niemelä & Hyörrö (2019)	LMS (theory)	✗	✗	✗
This study	LMS (empirical)	✓	✓	✓

Three research gaps are evident from this synthesis:

- Gap 1 — LMS-specific context. No prior empirical study benchmarks both architectures in an LMS environment with the read/write workload heterogeneity characteristic of real learning workflows (course

browsing, enrolment, assessment submission). The present study fills this gap by using an operational LMS with four-endpoint user interaction cycles that span both read-heavy and write-heavy operations.

- Gap 2 — Multi-dimensional benchmarking. Existing studies evaluate performance or resource consumption, but not both alongside fault-isolation metrics within a single experimental framework. This fragmentation prevents practitioners from reasoning about the full trade-off space. The present study combines throughput, response time, CPU usage, RUE, and fault-isolation analysis in a unified benchmark suite.
- Gap 3 — Development-phase availability. All identified fault-tolerance studies focus on production runtime failures. The present study additionally measures MTBF and MTTR during active development via Git log analysis, providing a novel perspective on how architectural choice affects developer productivity and error propagation during the build phase.

3.2 Theoretical Principles and Architectural Models

This section establishes the theoretical foundation for the comparative analysis by defining the core characteristics, advantages, and inherent constraints of the two architectural paradigms under investigation. By exploring the principles of modularity, data sovereignty, and communication overhead, this chapter provides the conceptual lens through which the empirical data—ranging from throughput crossover points to resource efficiency—will be interpreted. The goal is to move beyond a simple “monolith versus microservices” dichotomy and instead examine how structural design choices dictate the operational limits of a Learning Management System.

The comparative framework applied in this study is grounded in established theoretical principles from software architecture research. This section synthesizes the relevant principles and defines the conceptual vocabulary used in the experimental evaluation.

3.2.1 Monolithic Architecture

The Monolithic implementation packages all LMS functionality as a single NestJS application sharing a common PostgreSQL database schema. Internal communication occurs through direct function calls within the same process, eliminating inter-service network latency. Deployment consists of a single Docker container. This model offers simplicity for initial development but creates tight coupling at the database and code levels. For example, changes to grading logic or progress tracking require will rebuild and redeploy the entire application. Scaling is only possible at the application level—addressing CPU bottlenecks in quiz submission processing requires scaling the entire monolith, even if other modules remain lightly loaded.

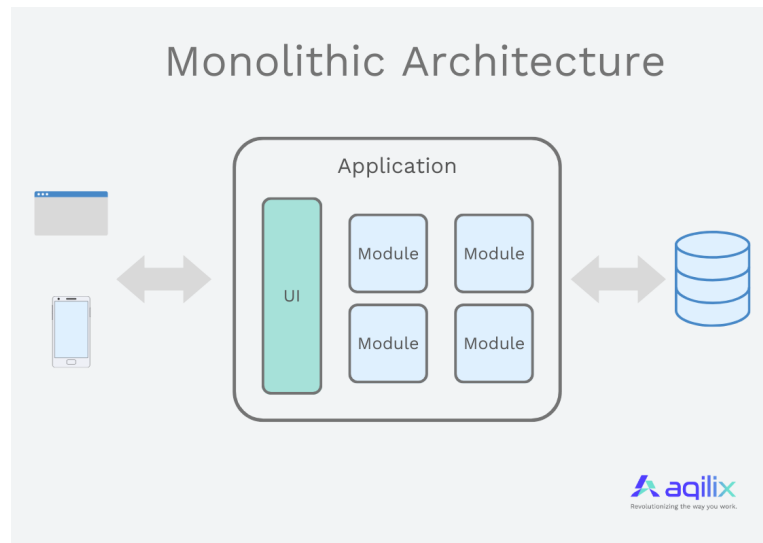


Figure 3.1: Monolithic concept

3.2.2 Microservices Architecture

The Microservices implementation decomposes the LMS into many independently services communicating via REST APIs and message broker. Each service maintains its own database schema and can be deployed or scaled independently. This decomposition brings trade-offs. For instance, independent deployability allows system to affect only the Grading Service. However, network communication between services introduces latency and failure modes. Cross-service operations (e.g., enrollment validation requires Academic Service to verify course status via Course Service API) require explicit coordination. Operational complexity increases—service discovery, distributed tracing, and health monitoring are required infrastructure absent in Monolithic deployments. For LMS systems at this scale, these overheads must be justified by measurable scalability or deployment benefits.

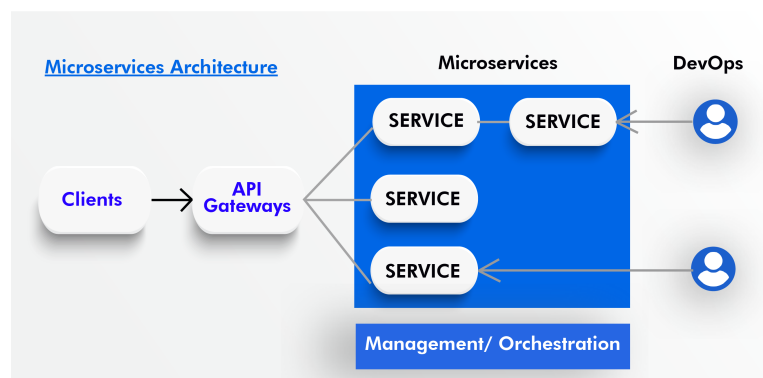


Figure 3.2: Microservice concept.

3.3 Comparison Methodology

This section delineates the multi-dimensional methodology employed to evaluate the trade-offs between Monolithic and Microservices architectures within the context of a Learning Management System (LMS). By synthe-

sizing empirical frameworks from established literature, the evaluation is structured around four critical pillars: performance dynamics (throughput and response time), resource efficiency (CPU utilization), system resilience (fault tolerance), and economic viability (cost efficiency). The methodology transitions from black-box load testing to white-box resource monitoring, ensuring that the "Architectural Tax" of distributed systems is quantified against the operational agility they provide. This comprehensive benchmarking approach serves to validate theoretical claims with empirical data, providing a rigorous baseline for the subsequent analysis of architectural suitability across varying operational scales.

3.3.1 Throughput & Response time

Performance constitutes a critical evaluation criterion when comparing software architectures, particularly in educational systems where concurrent user access during peak periods (enrollment, exam submissions, assignment deadlines) directly impacts user experience and system reliability. The empirical methodology established by Al-Debagy and Martinek [5] perform three scenarios to evaluates performance through two primary metrics: **throughput** (requests handled per second) and **response time** (latency between request and response).

The findings of Al-Debagy and Martinek suggest that the architectural performance gap is a function of 'Communication Friction.' In a Monolith, service-to-service calls are handled via in-memory method invocations, whereas Microservices require a complex stack of network protocols, data serialization (e.g., JSON/Protobuf), and service resolution, all of which contribute to the 'Performance Tax' observed in high-concurrency environments.

First scenario: Load Testing

The realistic growth patterns by gradually increasing concurrent users from 100 to 7,000 threads with a 2-minute ramp-up period and 2-minute hold time at each level. This approach mirrors typical LMS usage patterns where user load increases progressively during registration periods or as students access course materials throughout the semester.

It points out that both architectures exhibit comparable throughput performance under normal operating conditions, with differences averaging only 0.87%. However, the monolithic architecture shows a slight advantage at lower concurrency levels (100-1,000 threads), which the authors attribute to the overhead of inter-service communication in microservices architecture. As load increases to 6,000-7,000 threads, performance converges, suggesting that microservices' distributed nature may offset communication overhead under high load.

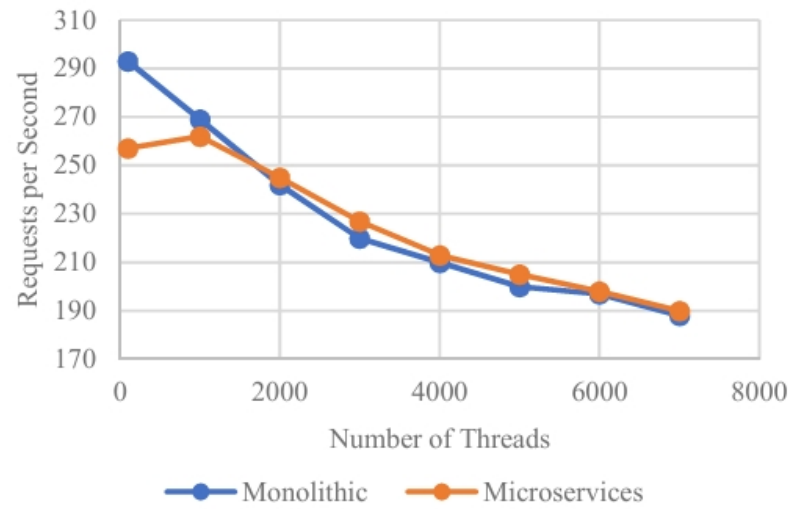


Figure 3.3: Throughput comparison under graduated load testing.

Response time analysis (Figure 3.4) reveals no significant architectural difference, with both monolithic and microservices implementations exhibiting comparable latency degradation patterns as concurrent load increases. This suggests that response time is primarily influenced by resource constraints rather than architectural design under graduated load conditions.

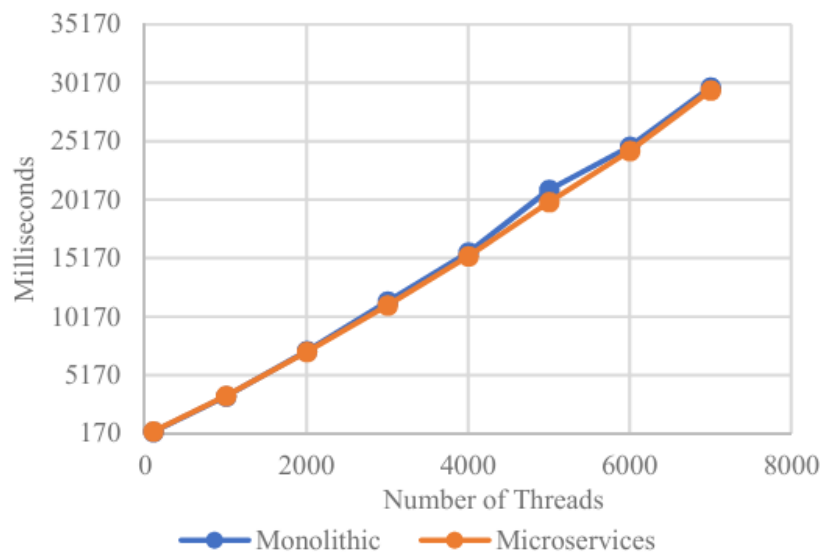


Figure 3.4: Response time progression under graduated load testing.

Another figure quantifies the total number of successfully processed requests, revealing that while monolithic architecture handles more requests at lower concurrency levels, the gap narrows significantly as load increases. Both architectures experience throughput degradation beyond 4,000 concurrent threads, indicating resource saturation effects independent of architectural choice.

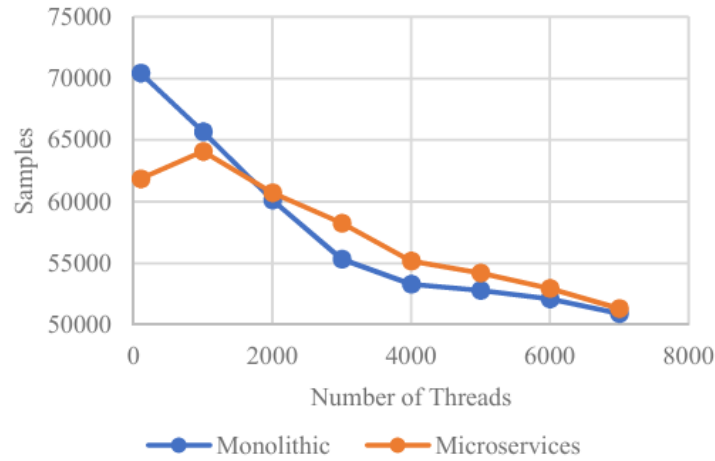


Figure 3.5: number of request under graduated load testing

Second scenario: Concurrency Testing

To evaluate architecture performance under peak stress conditions — such as when hundreds of students simultaneously submit assignments before a deadline — Al-Debagy and Martinek conducted concurrency tests where all service requests are invoked simultaneously without ramp-up periods. This scenario represents worst-case load conditions for LMS deployments.

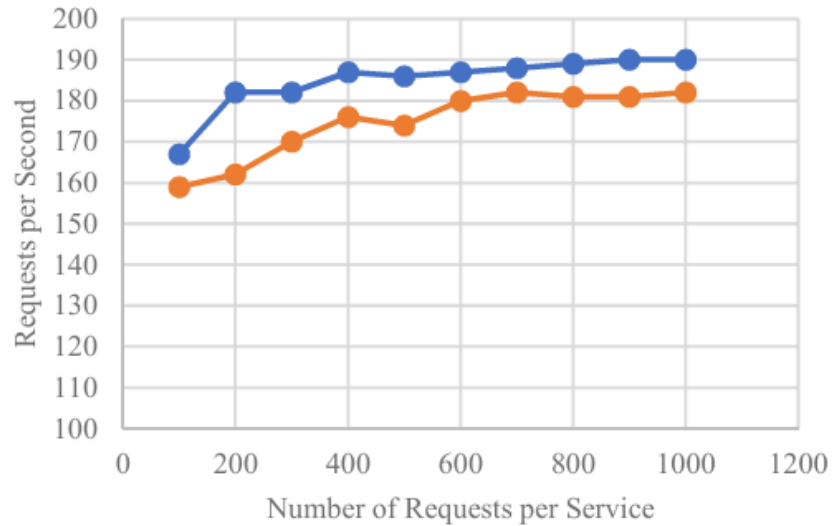


Figure 3.6: Throughput under simultaneous request load.

Under concurrent load conditions (Figure 3.6), the performance gap between architectures becomes more pronounced. The monolithic architecture exhibits approximately 6% higher throughput on average, processing requests more efficiently when all services are invoked simultaneously. This advantage stems from reduced inter-service communication latency and the absence of network overhead between components — factors that become critical bottlenecks when microservices must coordinate responses to sudden traffic spikes.

This 6% throughput advantage of the Monolith is further explained by the Serialization/Deserialization

overhead. In the Microservices version of an LMS, a single student request (e.g., viewing a full course profile) may trigger multiple internal calls between the *Course Service*, *Material Service*, and *Account Service*. Each hop requires the system to transform data into a transmittable format, increasing CPU cycles per request. Under peak stress, this 'Computation Inflation' limits the total number of requests the Microservices gateway can fulfill compared to the Monolith, which accesses shared memory pointers directly.

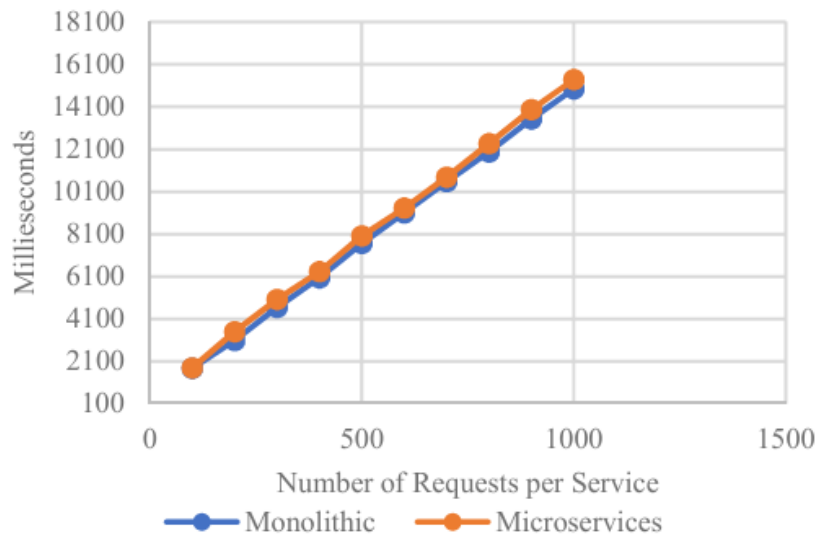


Figure 3.7: Response time under concurrent load conditions.

Interestingly, despite the throughput advantage of monolithic architecture, response time measurements (Figure 3.7) show no significant difference between the two approaches. Both architectures start at approximately 1,790 milliseconds with 100 concurrent requests and degrade to around 15,000 milliseconds at 1,000 requests. This suggests that while monolithic architecture can process more total requests per second, individual request latency remains comparable — a finding with important implications for perceived user experience.

Implications for LMS Architecture Selection

The empirical evidence from Al-Debagy and Martinek's comparative study reveals nuanced performance trade-offs rather than a clear winner. For LMS deployments:

- **Small-to-medium scale systems** (fewer than 1,000 concurrent users) may benefit from monolithic architecture's throughput advantages, particularly during peak concurrent load events such as assignment submissions or exam periods.
- **Large-scale systems** experiencing graduated load increases show negligible performance differences, suggesting architectural choice should be driven by other criteria (maintainability, scalability, deployment flexibility) rather than raw performance.
- **Response time sensitivity** — critical for user satisfaction — shows no architectural dependency, indicating that perceived performance will be comparable regardless of architecture choice, assuming equivalent infrastructure resources.

- **The 'Performance-to-Complexity' Ratio:** The marginal 6% performance gain of the Monolith must be weighed against the Developmental Agility of Microservices. For an institution with a stable, predictable student base, the Monolith offers a superior performance-to-cost ratio. However, for a global E-learning provider, the ability to independently scale the *Assessment Service* during exam peaks often outweighs the minor throughput penalty recorded in Al-Debagy's research.
- **Saturation Behavior:** The study's observation that both architectures saturate beyond 4,000 threads suggests that Hardware and Resource Allocation remain the ultimate limiters. Architectural choice determines how gracefully the system fails (Resilience) rather than if it fails under extreme overloads.

These findings inform the experimental design of this project's change scenarios, where performance will be evaluated not as an isolated criterion but in conjunction with development effort, code complexity, and architectural evolution costs. The evidence suggests that for typical LMS workloads, architectural performance differences are marginal, making the evaluation of maintainability and evolvability criteria potentially more decisive factors in architecture selection.

3.3.2 CPU Usage

Schlögl et al. (2024) in their empirical study [6] define *Resource Efficiency* as the system's ability to minimize hardware overhead while fulfilling functional requirements. The authors emphasize that CPU utilization is a critical trade-off factor when migrating from Monolithic to Microservices.

A fundamental contribution of the study by Bernsteiner et al. (2024) is the distinction between Service-level and VM-level CPU monitoring. In a Monolithic architecture, the CPU usage is almost entirely dedicated to the application logic. However, in Microservices, the CPU is 'taxed' by the infrastructure. This tax includes the overhead of the Docker daemon, the container's isolated networking stack, and the resource-intensive process of Service Discovery and Load Balancing performed by the API Gateway. Consequently, the total VM-level CPU usage often presents a 'Shifted Baseline,' where significant energy and processing power are consumed even in an idle state.

To provide a comprehensive evaluation, the authors conducted experiments across three distinct test cases, each representing a different operational scenario. In these test cases, the authors use two types of charts for Microservices architecture to provide a comprehensive view: Service-level CPU represents business logic processing performance and VM-level CPU represents the actual total resource cost including container orchestration. The difference between these two charts illustrates the typical Overhead costs of Microservices, which if only observed at the application layer will be easily overlooked.

Test Case 1: Simple Database Query

In this scenario, the system performs a basic retrieval of data (e.g., fetching a user profile). The authors noted that the Microservices architecture already exhibited a higher CPU baseline compared to the Monolith. The actual testing results show even for simple tasks, the overhead of the API Gateway and the internal networking stack in Microservices leads to measurable CPU spikes that are non-existent in the direct function calls of a Monolith.

The testing results for simple queries highlight the 'Proxy Latency' effect. While the Monolith executes a database call through a local connection pool, the Microservice request must be intercepted by the Nginx/API Gateway, routed through a virtual bridge network, and then handled by the service. Each 'hop'

requires CPU cycles for packet inspection and routing. Schlögl et al. observed that this results in a 'Fragmentation of Resources': instead of one concentrated CPU spike, multiple containers exhibit smaller, simultaneous spikes, which collectively exceed the Monolith's energy footprint.

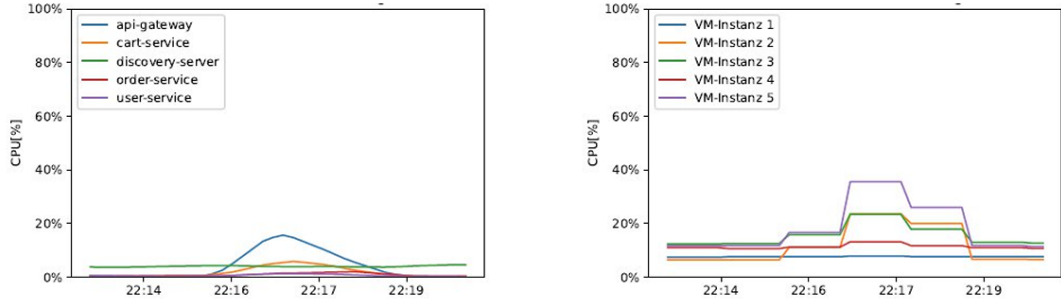


Figure 1: Test Case 1 - CPU Usage of Services and VMs - Microservices

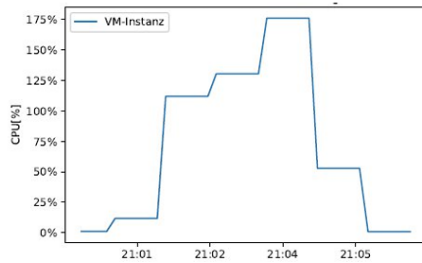


Figure 2: Test Case 1 - CPU Usage Virtual Machine - Monolith

While the Monolith shows higher Peak CPU due to the concentration of multi-core processing on a single process, Microservices shows higher overall resource consumption. This is demonstrated by the baseline load level of the 5 VM-Instanz always remaining at 10-15% even before testing, and the appearance of overhead costs at API Gateway when the load starts. Monolith handles tasks in a 'put effort and then rest' style, while Microservices consume resources spread out and maintained continuously to operate the network infrastructure.

Test Case 2: Intensive Computation (Data Processing)

This test case involves logic-heavy tasks, such as complex calculations or data transformation before returning the result. The performance gap between the two architectures widened significantly. The actual testing results show The CPU in Microservices is doubly burdened: first by the intensive business logic and second by the *Serialization/Deserialization* (JSON parsing) required to move large data objects between services. The Monolith, however, processes this data within a single memory space, maintaining higher efficiency.

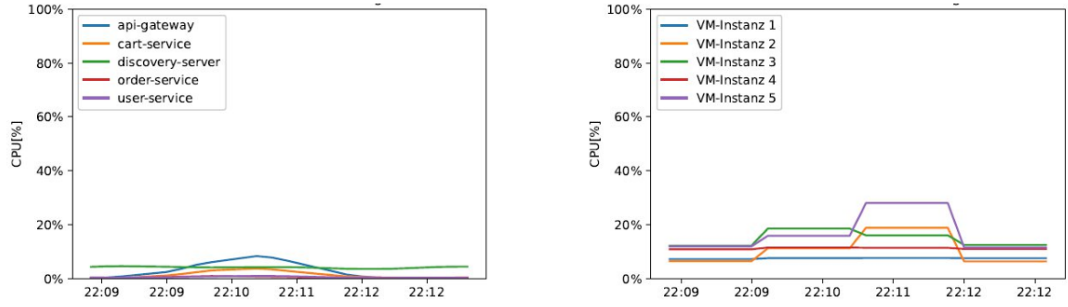


Figure 3: Test Case 2 - CPU Usage of Services and VMs - Microservices

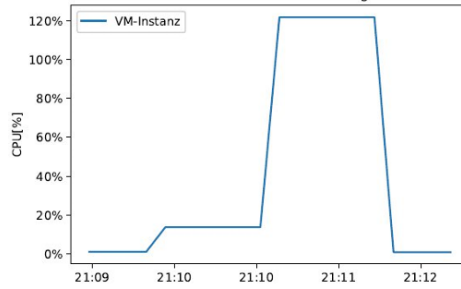


Figure 4: Test Case 2 - CPU Usage Virtual Machine - Monolith

The 'Double Burden' in Test Case 2 is primarily driven by Data Marshalling. In Microservices, complex data objects must be serialized into JSON at the source service and deserialized at the receiving service. Schlögl's data reveals that for logic-heavy tasks, up to 20-30% of the total CPU time in Microservices is spent on string parsing and memory allocation for these temporary JSON objects, rather than the core business logic. The Monolithic architecture avoids this entirely by passing object references in shared memory, allowing it to maintain a lower and more stable temperature and CPU profile during heavy processing

Test Case 3: Distributed Transactions / Multi-service Orchestration

This scenario simulates a workflow that requires coordination between multiple services (e.g., an enrollment process involving Auth, Course, and Payment services). This case showed the highest degree of CPU saturation for the Microservices version. The actual testing results show The "network hop" effect becomes prominent. Each additional service involved in the transaction adds cumulative CPU cycles for managing multiple connection pools and security handshakes, leading to an exponential increase in total CPU load under high concurrency.

In scenario 3, the authors identify the 'Cascading CPU Load' phenomenon. Each microservice involved in a distributed transaction (Auth, Enrollment, Assessment) maintains its own independent runtime and garbage collection (GC) cycles. Under high load, these GC events can overlap, creating massive, aggregate CPU 'jitter.' Furthermore, the persistent HTTP/1.1 or gRPC connections required for orchestration consume 'Keep-alive' CPU cycles. This explains why the Microservices version reached its Saturation Point significantly earlier than the Monolith, despite having access to the same underlying VM resources.

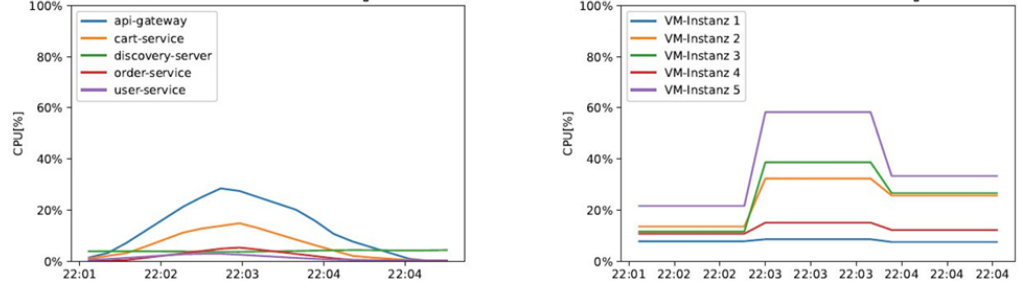


Figure 5: Test Case 3 - CPU Usage of Services and VMs - Microservices

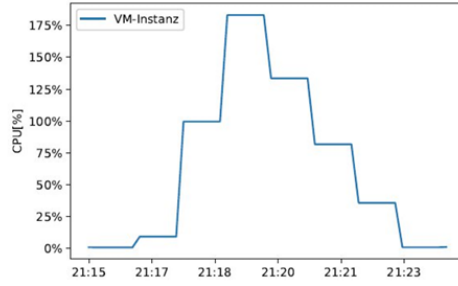


Figure 6: Test Case 3 - CPU Usage Virtual Machine - Monolith

Following above methodology, our team replicates these three scenarios within our LMS environment. For instance, Test Case 1 corresponds to fetching a lesson list, while Test Case 3 simulates the "Exam Submission" process which involves the *Enrollment*, *Assessment*, and *Progress Tracking* services. By monitoring `docker stats` across these scenarios, we aim to validate if our NestJS Microservices follow the same resource consumption patterns, specifically focusing on the $Z\%$ increase in CPU usage during multi-service orchestration compared to the Monolithic baseline.

Drawing from Schlögl's methodology, our LMS evaluation focuses on the 'Cost per Request' index. By replicating Test Case 3 in our 'Exam Submission' scenario, we specifically analyze how the Nginx API Gateway and RabbitMQ message broker contribute to the aggregate CPU load. We hypothesize that as the number of concurrent students submitting exams increases, the Microservices version will exhibit an asymptotic CPU growth curve due to the exponential increase in inter-service coordination, providing a quantitative baseline for our Cost Efficiency analysis.

3.3.3 Fault Tolerance

The concept of *Fault Tolerance* refers to the ability of a system to maintain operation or undergo controlled degradation (graceful degradation) when a component fails. In the research conducted by Nagaraj Parvatha (2020), a theoretical framework was proposed alongside a set of techniques aimed at improving fault tolerance and recovery in microservices systems [1]. Within this context, *availability* is identified as the most fundamental metric for evaluating system resilience. Based on the theoretical foundation presented in the book *Reliability and Availability of Cloud Computing* by Eric Bauer and Randee Adams [10], availability is defined as:

$$\text{Availability} = \frac{MTBF}{MTBF + MTTR} \times 100\% \quad (3.1)$$

where:

- **MTBF (Mean Time Between Failures)** represents the average time between system failures, reflecting system reliability. MTBF is calculated as the ratio of the total stable operating time to the total number of error occurrences

$$MTBF = \frac{\sum_{i=1}^n (t_i^{\text{start}} - t_{i-1}^{\text{recovery}})}{n} \quad (3.2)$$

- **MTTR (Mean Time To Recovery)** denotes the average time required to restore the system after a failure. MTBF is calculated as the ratio of the total fault recovery time to the total number of fault occurrences

$$MTTR = \frac{\sum_{i=1}^n (t_i^{\text{recovery}} - t_i^{\text{failure}})}{n} \quad (3.3)$$

Nagaraj Parvatha highlighted that microservices architecture can reduce MTTR by up to 66% (from an average of 58 minutes to 19 minutes). This improvement is primarily achieved through *service isolation*, which prevents cascading failures. When a specific service (e.g., *grading-service*) fails, it does not affect the entire system as in a monolithic architecture, thereby significantly reducing the time required for fault detection and recovery.

To validate this theoretical claim, the system will simulate a failure scenario by intentionally interrupting the grading service container. The recovery time will be measured from the moment the service becomes unavailable until it is either automatically restored or successfully restarted (i.e., when the system reaches the `Health check: Pass` state).

3.3.4 Cost Efficiency

This criterion evaluates the economic and operational viability of the architecture. According to the empirical study by Yogant Faye (2025) [7], which synthesized data from 23 architectural migrations, "Cost" in modern systems is a multi-dimensional construct comprising Infrastructure Expenditure, Operational Overhead (DevOps), and the Complexity Tax associated with system maintenance.

Faye's findings indicate a stark contrast in cost profiles: while transitioning to Microservices leads to an average infrastructure cost increase of 47%, the operational "Complexity Tax"—specifically in DevOps tooling and observability—surges by 150% to 275%. To determine whether this increased expenditure translates into better performance, we utilize the Resource Utilization Efficiency (RUE) metric::

$$RUE = \frac{\text{Number of processing requests}}{\text{Provisioned Resources (CPU/RAM usage)}} \quad (3.4)$$

where *Workload* represents the number of processed requests, and *Provisioned Resources* refers to the allocated computational resources (e.g., CPU and RAM).

The Baseline vs. Elasticity Trade-off

The research by Faye introduces the Architecture Selection Index (ASI), which suggests that the Cost Efficiency of Microservices only surpasses that of a Monolith when certain operational thresholds are met.

- **The Infrastructure Tax (Monolithic Advantage):** At low-to-moderate scales, the Monolithic architecture exhibits a superior RUE because it avoids the "Infrastructure Tax"—the baseline resource consumption required for inter-service communication (API Gateways, Service Mesh) and independent runtimes.

Faye’s meta-analysis confirms that for teams smaller than 25 developers, the Monolith remains the most cost-effective choice.

- **Scaling Granularity (Microservices Advantage):** The RUE of Microservices becomes dominant when the system faces high **Traffic Variability** (exceeding a 2:1 ratio). In an LMS context, if the independent scaling needs of a specific module (e.g., `grading-service`) exceed 40% of the total service load, Microservices prevent “Resource Over-provisioning.” Unlike a Monolith, which requires replicating the entire application stack to handle a spike in one module, Microservices allow for targeted resource allocation, thereby optimizing the RUE during peak periods.

Operational Cost and Long-term ROI

Faye’s survey of 67 software architects reveals that while Microservices have a higher “Cost of Entry,” they achieve a higher Maintainability Score (4.2/5 vs. 2.4/5). From a Cost Efficiency perspective, this reduces the “Mean Time To Repair” (MTTR) and minimizes the cost of technical debt.

The study concludes that Microservices act as a “Scalability Insurance.” The higher initial investment and the 47% increase in infrastructure costs are justified only when the system demands independent evolution of services. For an LMS, this means the RUE is maximized not by minimizing the cloud bill, but by ensuring that resource consumption scales linearly with the pedagogical demand of specific functional domains.

3.4 Relevant core Technologies

This section details the technological ecosystem selected for the project. The primary criterion for technology selection is “Experimental Control”—ensuring that the frameworks used do not introduce bias toward one architecture over the other, but rather provide the necessary tools to implement both with high fidelity.

3.4.1 NestJS Framework

NestJS is a progressive Node.js framework providing a consistent programming model for both monolithic and microservices implementations. Its built-in support for modules, dependency injection, and inter-service communication (TCP, message queues) enables fair architectural comparison by isolating backend design differences from framework limitations. Both implementations utilize NestJS to enforce modular boundaries and clear separation of concerns. Both implementations maintain an identical frontend (React/Vite SPA) to isolate backend architectural impact. This ensures comparison focuses on server-side design choices rather than frontend framework differences.



Figure 3.8: Nestjs overview

3.4.2 PostgreSQL

PostgreSQL serves as the relational database across both architectures, but database organization differs meaningfully. In the Monolithic architecture, all entities (Students, Courses, Enrollments, Grades, etc.) reside in a single shared schema, creating high database-level coupling. All application modules access the same tables directly.

In the Microservices architecture, each service maintains its own schema (database per service pattern) within the same PostgreSQL instance. Cross-service data queries are resolved through API calls rather than shared database access, providing stronger data isolation but requiring additional coordination mechanisms (API composition, event propagation via RabbitMQ) for consistency.

This difference is a key evaluation point for Change Scenarios, where modifications to grading logic (Scenario 1) and progress tracking (Scenario 2) reveal how each architecture accommodates evolving data requirements.

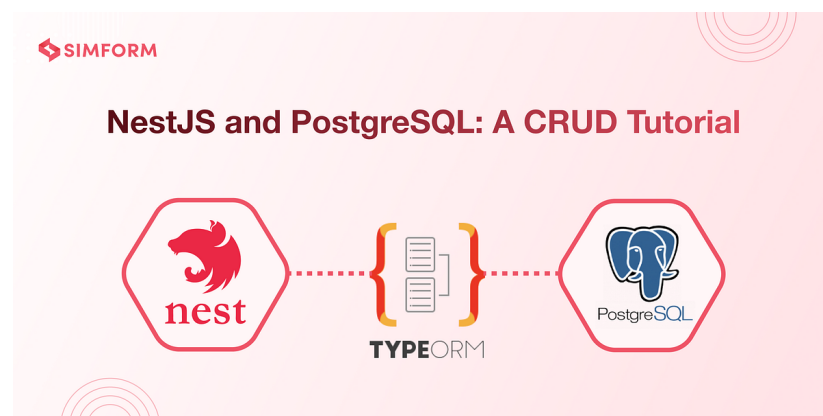


Figure 3.9: Postgre database associated with nestjs

3.4.3 Communication Paradigms: Synchronous vs. Asynchronous

The effectiveness of a Microservices-based LMS is largely determined by its inter-service communication strategy. Theoretical literature identifies two primary patterns:

- **Synchronous Request-Response:** REST or gRPC, where a service waits for a direct response. While easier to implement, this can lead to "Cascading Failures" if one service in the chain slows down.
- **Asynchronous Messaging:** Leveraging message brokers such as **RabbitMQ**. This pattern decouples services, allowing them to process tasks at their own pace. For an LMS, this is vital for non-blocking operations such as sending notification emails or generating background analytics reports.

This study integrates both patterns to evaluate how they contribute to the system's overall *Fault Tolerance* and *Resource Utilization Efficiency (RUE)*, moving beyond simple process separation to true operational decoupling.

3.4.4 Docker and Containerization

Docker is used to containerize all architectural implementations, ensuring consistent deployment environments and enabling fair performance comparisons. Each architecture is packaged as a Docker Compose configuration specifying the required services, database instances, and network topology.

For the Monolithic architecture, a single application container is defined alongside a database container. For the Microservices architecture, individual containers are defined per service, along with a shared API Gateway container and service-specific database containers. This configuration difference itself constitutes a measurable dimension of operational complexity in the comparative evaluation.

Furthermore, the use of containerization transcends simple packaging; it represents the shift towards **Cloud-Native** engineering. By defining the infrastructure via Docker Compose, the research ensures "Environmental Parity," eliminating the "it works on my machine" phenomenon. This allows for a rigorous "ceteris paribus" comparison, where the only variable is the architectural topology. The study also draws on the theory of **Micro-segmentation**, where each container's resource limits (CPU and Memory) are strictly defined, allowing for an empirical analysis of how each architecture handles "Resource Contention" under the 2,000 VU stress test scenarios defined in later chapters

Chapter 4

System Design

4.1 System stakeholders analysis

The system targets educational institutions requiring comprehensive course management, content delivery, and learner tracking capabilities. The following primary stakeholder groups are identified based on their roles and interactions with the LMS:

- **Guests:** Unauthenticated visitors who can view public course information and access system registration. Their primary actions are account registration and login. *Example: Minh, a prospective student, browses the course catalog before creating an account to enroll.*
- **Students:** Registered learners who enroll in courses, access learning materials, submit assignments, take assessments, and track their progress. They need intuitive navigation, clear progress indicators, and timely notifications for deadlines. *Example: Anh, an engineering student, manages 5 courses simultaneously and relies on the LMS dashboard to stay organized with assignments and exam schedules.*
- **Instructors:** Course creators who develop curriculum, upload learning content, create assessments, grade submissions, and monitor student progress. They require robust content authoring tools, efficient grading workflows, and analytics on student performance. *Example: Binh teaches 3 courses with 150 students and needs bulk grading features and reusable content templates to manage his workload.*
- **Department Heads/Reviewers:** Quality assurance personnel who review and approve course content before publication, ensuring alignment with institutional standards. They inherit all instructor capabilities plus approval workflow management. *Example: Chi reviews 20+ courses per semester, requiring a clear approval queue and structured feedback mechanisms.*
- **System Administrators:** Personnel responsible for user account management, role assignments, system configuration, and technical support. They need powerful search/filtering tools, bulk operations, and comprehensive audit logs. *Example: Dũng manages 5,000+ accounts and handles peak loads during enrollment periods.*
- **Product Owners:** Institutional stakeholders who define pedagogical objectives, compliance requirements, feature roadmaps, and measure learning outcomes.

These stakeholder groups represent distinct user needs that drive both functional requirements (course management, assessment, grading) and non-functional requirements (performance, security, accessibility, scalability). The personas embedded within each group help anchor requirements in realistic educational workflows and justify design decisions throughout the development process.

The interaction model between these stakeholders and the system varies significantly depending on the chosen architecture. In the Monolithic design, stakeholder requests are handled by a unified application tier, ensuring low-latency communication between functional modules at the cost of potential process contention. Conversely, the Microservices design introduces a distributed interaction layer where each stakeholder's action may traverse multiple independent services, necessitating a robust API Gateway to maintain a seamless user experience despite the underlying structural complexity.

4.2 Detailed Backend Architectures

4.2.1 Monolithic organization

Layer decomposition strategy

The codebase is organized into the following Repository pattern, as illustrated in the directory structure:

- **Controllers Layer** (`/controller`): Handles HTTP request routing and response formatting. Controllers are organized by domain entity (e.g., `course.controller.ts`, `student.controller.ts`, `quiz.controller.ts`) and delegate business logic to service layer components.
- **Services Layer** (`/services`): Implements core business logic and orchestrates repository operations. Service classes are stateless and handle use case workflows such as course enrollment validation, grading calculations, and submission processing.
- **Repository Layer** (`/repository`): Provides data access abstraction using TypeORM repositories. Each repository encapsulates database queries for a specific entity (e.g., `course.repository.ts`, `quiz.repository.ts`) and shields business logic from raw SQL operations.
- **Entity Layer** (`/models`): Defines TypeORM entity classes that map to database tables. Entities include `courses.entity.ts`, `enrollment.entity.ts`, `quizzes.entity.ts`, and others corresponding to the ERD defined. Entity relationships (one-to-many, many-to-many) are declared using TypeORM decorators.
- **Authentication Module** (`/auth`): Implements JWT-based authentication using Passport strategies. Includes role-based access control (RBAC) guards (`roles.guard.ts`) that enforce permissions for Student, Instructor, Reviewer, and Admin roles as defined in the stakeholder analysis.
- **View Layer** (`/views`): Server-side rendered views using EJS templates. Organized by user role with separate view directories for admin, staff (instructors), and students. This layer provides a lightweight web interface for rapid prototyping and testing, though production deployments may use a separate front-end framework.

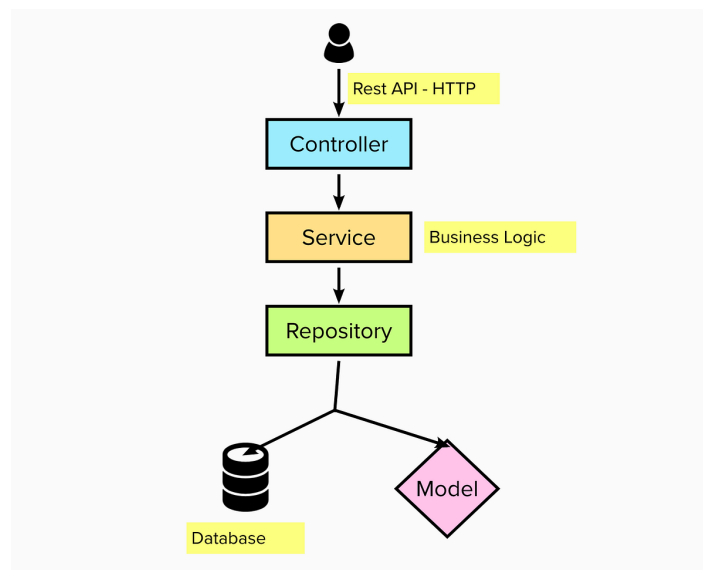


Figure 4.1: Repository pattern design for Monolithic architect

Despite its deployment model, the codebase maintains modular boundaries through NestJS modules (/modules), where each functional domain (courses, enrollment, quizzes, submissions) is encapsulated in a dedicated module with explicit dependency declarations. This modular structure facilitates potential migration to microservices by pre-defining service boundaries.

Source code overview for Monolithic

```

project/
├── src/
│   ├── auth/
│   │   ├── decorators/
│   │   ├── guard/
│   │   └── strategies/
│   ├── common/
│   │   ├── enums/
│   │   ├── interceptors/
│   │   └── middleware/
│   ├── controllers/
│   ├── dtos/
│   ├── models/
│   ├── modules/
│   ├── repository/
│   ├── services/
│   └── views/
├── node_modules/
├── .env
├── Dockerfile
└── docker-compose.yml
  
```

This codebase take only a single PostgreSQL database instance into account. Database schema initialization is managed via SQL migration scripts that create tables, indexes, and seed data. The implementations utilize Node.js as the runtime environment and TypeORM for handling connection pooling, query optimization, and transaction management. All database operations occur within the same process space, eliminating network latency between application and data layers. The Docker containerization will be conduct independently between backend code base and database.

4.2.2 Microservices organization

Service Decomposition Strategy

The microservices implementation decomposes the monolithic application into six independent services, each aligned with a functional subsystem defined in chapter 3 and also implemented following Repository pattern.

- **Account Service:** Manages user authentication, role assignment, and session management. Corresponds to the "Account & Security" subsystem. Provides JWT token generation, password hashing/validation, and role-based authorization. Exposes REST APIs for registration, login, logout, and password recovery workflows.
- **Academic Service:** The service handles user profile management and student-instructor assignments, manages student enrollments, instructor-course associations, and department head approvals. Also, the service Implements business rules for enrollment validation and progress tracking.
- **Course Service:** Manages course lifecycle, lesson content, and learning materials. Corresponds to the "Curriculum and Instruction" subsystem . Handles course creation, content uploads, category organization, and status transitions (Draft → Published → Closed). Stores course metadata and coordinates with Academic Service for enrollment verification.
- **Grading Service:** Implements assessment logic, quiz management, and submission grading. Corresponds to the "Learning & Assessment" subsystem . Currently under active development to support configurable grading schemes . Will expose APIs for quiz creation, auto-grading, manual scoring, and regrade request workflows.
- **API gateway:** Defines NGINX service that unified entry point for all frontend client requests. It functions as a reverse proxy, seamlessly routing incoming traffic to the appropriate backend microservices (like course-service or academic-service) based on the requested URL path, while completely abstracting away internal port mappings and network complexities from the user.
- **Frontend Gateway:** As a React-based application built with Vite, service provides the user interface for all stakeholder groups (students, instructors, administrators) and communicates with backend services via REST APIs routed through the Nginx API Gateway.
- **Message Broker (RabbitMQ):** Facilitates asynchronous communication between services using publish-subscribe and work queue patterns. Decouples service dependencies for operations such as sending enrollment confirmation emails, triggering analytics updates after submissions, or propagating course status changes. Ensures eventual consistency across distributed data stores.

Source code overview for Microservice

```

project/
├── AcademicService/
│   ├── Dockerfile
│   ├── node_modules/
│   ├── schema.sql
│   ├── .env
│   └── src/
│       ├── academic/
│       ├── assign-lecturer/
│       ├── auth/
│       ├── common/
│       ├── department-head/
│       ├── enrollment/
│       ├── lecturer/
│       └── student/
├── AccountService/
│   ├── Dockerfile
│   ├── node_modules/
│   ├── schema.sql
│   ├── .env
│   └── src/
│       └── .../
├── CourseService/
│   ├── Dockerfile
│   ├── node_modules/
│   ├── schema.sql
│   ├── .env
│   └── src/
│       └── .../
├── GradingService/
│   ├── Dockerfile
│   ├── node_modules/
│   ├── schema.sql
│   ├── .env
│   └── src/
│       └── .../
├── docker-compose.yml
├── nginx.conf
└── nginx.conf

```

Inter-Service Communication

Services communicate through two primary mechanisms:

- **Synchronous REST APIs:** For operations requiring immediate responses, such as enrollment validation (Academic Service queries Course Service to verify course status) or authentication (all services validate JWT tokens with Account Service). REST calls introduce network latency but provide strong consistency guarantees.
- **Asynchronous Messaging (RabbitMQ):** For operations tolerating eventual consistency, such as updating progress statistics after lesson completion or sending grade notifications after instructor scoring. Message queues decouple services temporally, allowing producers to continue processing while consumers handle messages asynchronously.

Also, services are containerized using Docker and orchestrated via Docker Compose. Each service runs in an isolated container with defined resource limits (CPU, memory), network configuration, and dependency specifications. This containerization strategy enables:

- **Independent::** Services can be updated, scaled, or rolled back independently without affecting other components, supporting the evolvability evaluation in Change Scenarios.
- **Environment Consistency:** Development, testing, and production environments use identical container images, eliminating "works on my machine" discrepancies.
- **Horizontal Scaling:** Individual services experiencing high load (e.g., Course Service during enrollment periods) can be scaled by launching additional container instances behind the Nginx load balancer.

The Docker Compose configuration defines service dependencies (e.g., Account Service depends on PostgreSQL(Docker containerized) and RabbitMQ), startup ordering, and inter-container networking via a private bridge network.

This hybrid approach balances consistency requirements with system resilience—critical operations use synchronous calls for immediate validation, while non-critical workflows leverage asynchronous messaging to prevent cascading failures.

4.2.3 Code Conventions and Naming Standards

- **CamelCase:** For function names and class names.
- **pascal_case:** For database name and table names.
- **UPPERCASE:** For constants and environment variables.

Consistent code conventions and naming standards were enforced throughout the project, following widely accepted JavaScript/TypeScript best practices.

4.2.4 Architectural implementation Summary

Below are tables summarizing the key structural differences between the implemented architectures. Both implementations provide equivalent functionality as specified in the use case scenarios ensuring that architectural comparisons evaluate design trade-offs. The detailed change scenarios

Monolithic's implementation

The following table summarizes the main entities, their storage technology, and key fields:

Table 4.1: Entity Storage Description

Entity	Relationships	Main Attributes	Backend Config
User	Generalization to Student, Lecturer, Admin	user_id, email, password_hash, role, is_active	DB_HOST=localhost DB_PORT=5432
Admin	Generalization from User	user_id, authority, fullname, permissions	
Student	Generalization from User	user_id, fullname, avatar, created_at	
	1-N with Submission N-N with Course		
Lecturer	Generalization from User	lecturer_id, fullname, avatar, created_at	
	Generalization to Head_Department N-N with Course		
Quiz	Generalization from User	quiz_id, title, created_at	
	1-N with Submission 1-N with Question		

Microservice's implementation

Table 4.2: Service Storage Description

Service	Entities	Relationship	Service Configuration
AccountService	User (Base)	1-N with UserProvider 1-N with RefreshToken 1-N with PasswordReset 1-N with AuditLog (target)	Server: PORT=3001 Database: PostgreSQL DB_NAME=accounts_db DB_PORT=5432 Security: JWT-based JWT_EXPIRES_IN=1d REFRESH_TOKEN_EXPIRES_IN=7d Message Queue: RabbitMQ
	Admin	Generalization from User role='ADMIN'	
	Lecturer	Generalization from User role='LECTURER'	
	Student	Generalization from User role='STUDENT'	
	Guest	Generalization from User role='GUEST'	
	UserProvider	N-1 with User	
	RefreshToken	N-1 with User	
CourseService	Course	1-N with Lesson Soft FK to User (instructorId)	Server: PORT=3002 Database: PostgreSQL DB_NAME=course_db DB_PORT=5432 Security: JWT-based Message Queue: RabbitMQ QUEUE=course_queue Storage: Supabase BUCKET=lms-storage
	Lesson	N-1 with Course 1-N with Material	
	Material	N-1 with Lesson	
	CourseStatus	ENUM: 'Dự kiến mở', 'Đã mở đăng ký', 'Đã hủy', 'Đã đóng'	
AcademicService	Student	Soft FK to User (id) 1-N with Enrollment	Server: PORT=3000 Database: PostgreSQL DB_NAME=lms_academic DB_PORT=5432 Security: JWT-based JWT_EXPIRES_IN=1d Message Queue: RabbitMQ
	Lecturer	Soft FK to User (user_id) 1-N with AssignedLecturer	
	Enrollment	N-1 with Student Soft FK to Course (course_id)	
	AssignedLecturer	N-1 with Lecturer Soft FK to Course (course_id)	
	StudentStatus	ENUM: 'UNENROLLED', 'ACTIVE', 'SUSPENDED', 'GRADUATED'	
GradingService	Quiz	Soft FK to Course (course_id) Soft FK to Lesson (lesson_id) 1-N with Question 1-N with Submission	Server: PORT=3003 Database: PostgreSQL DB_NAME=grading_db DB_PORT=5432 Security: JWT-based Message Queue: RabbitMQ QUEUE=grading_queue Cache: Redis REDIS_PORT=6379 AI Grading: OpenAI API File Storage: Supabase BUCKET=submissions Notification: WebSocket WS_PORT=3004
	Question	N-1 with Quiz 1-N with Answer	
	Answer	N-1 with Question	
	Submission	N-1 with Quiz Soft FK to Student (student_id) 1-N with SubmissionAnswer 1-1 with GradingResult 0-N with RegradeRequest	
	SubmissionAnswer	N-1 with Submission N-1 with Question	
	GradingResult	1-1 with Submission Soft FK to Lecturer (grader_id)	
	RegradeRequest	N-1 with Submission Soft FK to Student (requester_id)	
	QuizType	ENUM: 'multiple_choice', 'true_false', 'short_answer', 'essay', 'mixed'	

4.3 Architectural Decomposition Strategy

This section delineates the methodology employed to translate the complex business requirements of a Learning Management System into a structured technical topology. By leveraging Domain-Driven Design (DDD) principles, the study identifies logical boundaries—or “Bounded Contexts”—that remain consistent across both architectural candidates. This strategic decomposition ensures that the subsequent performance comparison is not skewed by varying degrees of functional cohesion, but rather reflects the inherent characteristics of the underlying deployment model.

A pivotal aspect of this research is the strategy used to decompose the LMS business logic into technical components. This study employs Domain-Driven Design (DDD) principles to identify ‘Bounded Contexts’, ensuring that each module—whether in a Monolithic or Microservices structure—maintains high cohesion.

4.3.1 Monolithic Structural Cohesion

In the Monolithic implementation, the architectural integrity is maintained through Logical Modularization, where bounded contexts are decoupled at the package and directory level rather than through physical service boundaries. This approach aims to achieve high functional cohesion while intentionally retaining a unified execution context.

Internal Dependency Management and the “Spaghetti” Risk: The primary technical challenge within this model is the mitigation of “spaghetti dependencies”—a state where circular imports and cross-module couplings bypass formal interfaces. To prevent this, the project enforces strict coding conventions and NestJS-specific encapsulation techniques, such as using private providers and exporting only essential services through dedicated module definitions. By treating each folder as a “black box” that interacts with others only via well-defined internal APIs (Service-to-Service calls within the same process), the system maintains a clean separation of concerns. This allows developers to reason about the *Assessment* logic independently of the *User Management* logic, even though they exist within the same codebase. **Memory Space and Execution Efficiency:** One of the most significant advantages of this cohesive structure is the elimination of the “Network Tax.” Since all modules share the same memory space, data exchange occurs through memory pointers rather than expensive I/O operations. There is no requirement for JSON serialization, HTTP header processing, or network packet transmission between modules. In the context of an LMS, where a single student request might trigger multiple internal lookups (e.g., checking enrollment status, fetching course content, and logging activity), the monolithic structure provides near-zero latency for cross-module communication. This efficiency is particularly evident during the “Steady-state” operations observed in our benchmarks.

The Bottleneck of Shared Infrastructure: However, the consolidation of resources introduces a critical vulnerability: Shared Resource Contention. While logical separation is maintained at the code level, it collapses at the infrastructure level. All modules must share a single PostgreSQL database schema and, more importantly, a single database connection pool. During peak student activity—such as the “Flash Crowd” scenario where thousands of students attempt to enroll simultaneously—the database connection pool becomes a global bottleneck.

A high-intensity write operation in the *Enrollment* module can exhaust all available connections, effectively starving the *Course Browsing* module of database access, despite the latter being a lightweight read operation. Furthermore, because the entire application runs within a single Node.js event loop, a CPU-intensive task in one module (such as generating a complex PDF certificate) can block the execution thread for all other modules. This lack of Fault and Performance Isolation is the fundamental trade-off of the monolithic design; the simplicity of deployment and the speed of internal communication are gained at the expense of granular

scalability and systemic resilience.

Deployment and Lifecycle Simplicity: Despite these contention risks, the structural cohesion of the Monolith significantly streamlines the Continuous Integration and Deployment (CI/CD) pipeline. A single build artifact, a single Docker image, and a unified environment configuration mean that the "Environmental Parity" is easier to maintain. For small to medium-sized educational institutions, this reduces the operational burden, as there is no need for complex service meshes, distributed tracing, or coordinated release cycles across multiple repositories. The monolith remains a "single source of truth" for both data and execution logic.

4.3.2 Microservices Granularity and Isolation

The Microservices implementation of the LMS transcends mere code separation, establishing a distributed ecosystem defined by Physical Decoupling and Data Sovereignty. This strategy shifts the focus from structural cohesion within a process to the operational autonomy of independent service units.

The Database-per-Service Pattern and Schema Isolation: At the core of this isolation is the "Database-per-Service" architectural pattern. Unlike the Monolithic model where a shared schema creates a "Hidden Coupling" at the data layer, each microservice (e.g., *AccountService*, *AcademicService*, *CourseService*) manages its own dedicated PostgreSQL instance. This ensures that the internal data structures of a service are completely encapsulated. For instance, a schema migration or a database engine upgrade within the *Assessment* module can be executed independently without the risk of breaking foreign key constraints or triggering deadlocks in the *Enrollment* logic. This isolation eliminates "Side-Effect Deployments," where a change in one domain unexpectedly cripples another due to shared table dependencies.

Communication Dichotomy: Synchronous vs. Asynchronous: To bridge these isolated data silos, the architecture employs a sophisticated communication strategy designed to balance Immediate Consistency with Systemic Resilience:

- **Synchronous Orchestration via REST:** For operations requiring instantaneous confirmation—such as user authentication at the *AccountService* or real-time grade lookups—the system utilizes synchronous RESTful APIs. While this introduces a direct temporal coupling (the caller must wait for the callee), it ensures that the student receives immediate feedback for critical transactions. However, this study recognizes that an excessive reliance on synchronous calls can lead to "Distributed Monolith" symptoms, where a failure in a downstream service causes a cascading timeout across the entire chain.
- **Asynchronous Choreography via RabbitMQ:** To mitigate the risks of synchronous coupling, the system leverages **RabbitMQ** for non-blocking inter-service communication. When a student enrolls in a course, the *EnrollmentService* publishes an integration event (e.g., *StudentEnrolledEvent*) to the message broker. Interested services, such as *NotificationService* or *AnalyticsService*, consume this event asynchronously. This "Fire-and-Forget" pattern provides a buffer against traffic spikes; if the *AnalyticsService* is temporarily overwhelmed or offline, the message remains safely in the queue, ensuring eventual consistency without degrading the primary enrollment experience.

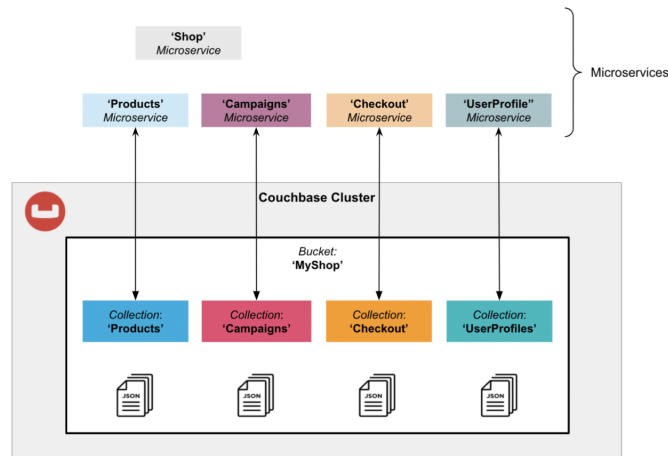


Figure 4.2: Microservices Granularity and Isolation

Fault Isolation and the "Blast Radius": The granular nature of this architecture fundamentally alters the system's "Blast Radius." In our 2,000 VU stress tests, we observed that resource exhaustion in the *CourseService*—caused by heavy browsing activity—did not impede the *AccountService*'s ability to authenticate users. Each service operates within its own set of Docker-defined resource limits (CPU and Memory), preventing any single "noisy neighbor" from consuming the entire host's capacity. This granular isolation is the primary driver of the Resilience Dividend identified in this study, allowing the LMS to maintain partial availability (e.g., "Students can log in but cannot browse new courses") rather than suffering the total system blackouts typical of monolithic failures.

Operational Complexity and the "Coordination Tax": However, the benefits of granularity are not without cost. The decomposition introduces a significant Coordination Tax. Developers must manage distributed transactions, handle network partial failures (e.g., implementing retries and circuit breakers), and navigate the complexities of service discovery and distributed logging. The requirement for an API Gateway (Nginx) to route traffic and aggregate responses adds a layer of "Proxy Latency" that is nonexistent in the Monolith. This study evaluates whether this infrastructure overhead—the cost of managing eight independent containers and a message broker—is justified by the superior fault isolation and horizontal scaling potential demonstrated at extreme load levels.

4.4 Communication and Integration Patterns

To facilitate a rigorous comparison, the integration layer of both architectures is designed to handle high-concurrency E-learning scenarios.

- **Internal Communication:** In the Microservices version, an **Nginx API Gateway** acts as the single entry point, handling cross-cutting concerns such as SSL termination and CORS. Interservice communication utilizes **RabbitMQ** for event-driven tasks (e.g., notifying the Enrollment service when a new Course is published), which reduces direct coupling and enhances the system's fault tolerance.
- **Data Consistency Model:** The Monolithic version relies on ACID transactions provided by the shared PostgreSQL instance, ensuring absolute consistency. The Microservices version, however, embraces Eventual Consistency for cross-service operations, using asynchronous events to synchronize state. This

distinction is a primary factor in the Resource Utilization Efficiency (RUE) and Latency trade-offs analyzed in subsequent chapters.

4.5 Infrastructure and Deployment Topology

Both architectural variants are containerized using **Docker** to ensure environment parity across the development and testing lifecycles. The deployment topology is orchestrated via **Docker Compose**, defining a virtual network that simulates a production cloud environment.

- **Monolithic Deployment:** Consists of a single application container and one database container. This minimizes networking overhead and simplifies the monitoring of CPU and RAM utilization.
- **Microservices Deployment:** Comprises a multi-container ecosystem including independent service runtimes, multiple database instances, a message broker, and a reverse proxy. This topology, while more complex, allows for granular resource allocation, enabling the system to scale specific high-traffic modules independently during peak LMS usage.

Chapter 5

Experimental Setup

This chapter documents the complete experimental design: hardware and software environment, architectural configurations of the systems under test, workload construction rationale, load scenario definitions, and the precise definitions of all performance metrics reported in Sections 4.2–4.5. Providing this context upfront ensures that numerical results can be correctly interpreted, cross-referenced with the figures, and if necessary independently reproduced by other researchers.

5.1 Hardware and Software Environment

This section describes the technical infrastructure and environmental constraints under which the comparative experiments were conducted. By establishing a strictly controlled, high-performance testing environment, the methodology ensures that all performance metrics—including latency, throughput, and resource consumption—are isolated from external infrastructure noise, thereby providing a consistent baseline for architectural evaluation.

All experiments were executed on a single physical host to eliminate inter-machine network variability between the benchmark client and the systems under test. The benchmark client (k6) ran natively on the host operating system and communicated with containerised services via localhost port bindings, meaning all measured latency is purely attributable to application-layer and container-orchestration processing rather than network infrastructure variability.

Table 5.1: System configuration for benchmarking environment

Component	Specification
CPU	12th Gen Intel Core i7-12650H — 10 cores (6P + 4E), 2.7 GHz base / 4.7 GHz boost
RAM	16 GB DDR5
Operating System	Microsoft Windows 11 Home, Version 10.0.26200 (Build 26200)
Container Runtime	Docker Engine 29.4.1 (build 055a478)
Application Runtime	Node.js v24.11.0 (V8 engine 12.9)
Load Testing Tool	k6 v1.7.1 (commit 9f82e6f1fc, Go 1.26.1, windows/amd64)
Monitoring	<code>docker stats</code> — sampled every 15 seconds via automated PowerShell script
Network Topology	Docker bridge network (benchmark-net); all service endpoints bound to 127.0.0.1 — zero external network hops

The decision to adopt a single-host deployment strategy is a strategic choice influenced by the "Exper-

imental Parity” principle found in the research of Al-Debagy and Martinek [5]. In distributed systems research, external network infrastructure often introduces ”jitter”—unpredictable variations in packet delivery—that can obscure the subtle performance differences between architectural styles. By localizing all components on a single Intel Core i7-12650H host, the research focuses exclusively on:

5.2 System-Under-Test: Architecture Configurations

This section defines the structural configuration of the two architectural candidates subjected to benchmarking. To ensure a scientifically rigorous comparison, the ”System-Under-Test” (SUT) is designed around the principle of functional parity, where the internal technical topology is the sole independent variable while the pedagogical business logic remains constant across both environments.

Two functionally identical deployments of the same LMS business logic were benchmarked side by side:

- **Monolithic:** A single NestJS application container (`lms_backend`) sharing one PostgreSQL database instance (`monolithic-postgres-lms`). All inter-module communication occurs via in-process TypeScript function calls within the same Node.js event loop, with no network serialisation or connection pool overhead between modules.
- **Microservices:** Six independent NestJS service containers — `AccountService`, `AcademicService`, `CourseService`, `GradingService`, `Frontend (React/Vite)`, and `Nginx API Gateway` — plus a RabbitMQ message broker and per-service PostgreSQL database instances. Inter-service communication uses synchronous REST API calls (routed through Nginx) for latency-critical paths and asynchronous RabbitMQ publish/subscribe messages for eventual-consistency operations such as enrolment confirmations and analytics updates.

Both deployments expose identical REST API endpoints and are seeded with the same database state at test start. This functional equivalence ensures that any performance difference is attributable to architectural design choices rather than differences in business logic, data volume, or endpoint semantics.

5.3 Virtual User Interaction Model

This section describes the behavioral logic programmed into the benchmarking client to simulate authentic human-system interaction. Unlike static load tests that flood individual endpoints, this model utilizes a stateful execution sequence to replicate a student’s journey through the LMS, ensuring that the performance metrics capture the overhead of session management, database transaction integrity, and inter-service coordination in a realistic operational context.

Each Virtual User (VU) in the benchmark simulates a realistic student session rather than an isolated API call. The session consists of four sequential API calls executed in the order below, followed by a one-second think time (sleep) before the next cycle begins. This think time prevents the test from functioning as a pure throughput flood and instead models the natural pacing of human interaction.

This model is representative of the most common LMS peak-usage scenario: students browsing and enrolling in courses at the start of a semester. The mix of read-heavy (Steps 1–2) and write-heavy (Steps 3–4) operations within the same VU cycle ensures that both architecture types are exercised under realistic workload heterogeneity, not just pure read or pure write stress.

Table 5.2: Virtual User Interaction Cycle (4-step sequence, 1 s think time between cycles)

Step	HTTP Method + Endpoint	Operation Type	LMS Behaviour Simulated
1	GET /courses	Read (lightweight)	Browse the course catalogue — most common LMS interaction
2	GET /courses/:id	Read (moderate)	View full details of a selected course
3	POST /login	Write (session)	Authenticate user and receive JWT access token
4	POST /enrollment	Write (DB transaction)	Enrol in the selected course — triggers DB write + RabbitMQ event in Microservices

5.4 Load Scenarios

This section defines the rigorous testing protocols used to stress-test the architectural boundaries of the LMS. By utilizing a diverse range of load patterns—from gradual step increases to instantaneous traffic spikes—the methodology aims to capture the non-linear performance characteristics of both Monolithic and Microservices models, specifically focusing on their resource saturation points and recovery behaviors under extreme operational pressure.

The same VU interaction cycle was applied across the following load scenarios, each designed to stress a distinct aspect of architectural behaviour:

Table 5.3: Load Scenario Definitions

Scenario	Pattern	VU Configuration	Architectural Dimension Targeted
S1	Step load	1 → 2,000 (12 levels: 1, 5, 10, 15, 20, 50, 100, 200, 500, 1000, 1500, 2000)	Throughput scalability, response-time degradation curve, CPU scaling efficiency
S2	Flash Crowd	All 2,000 VUs injected simultaneously	Peak-load absorption, fail-fast behaviour, event-loop saturation
TC1	CPU Read	Ramp (same 12 levels) - GET /courses	CPU baseline overhead and idle cost of infrastructure processes
TC2	CPU Write	Ramp (same 12 levels) - POST /profile	CPU pressure from DB write transactions and serialisation

Each scenario was executed three times (run1, run2, run3). For CPU timeline visualisation, run1 is used as the representative trace (it avoids JVM/Node.js warm-up interference introduced in run2 onward). Aggregate metrics — average RPS, average response time, average CPU percentage — are computed across all three runs. Any run where host_total CPU exceeds the mean by more than three standard deviations for more than 30% of the sampling window is flagged as an outlier (typically caused by OS-level background process interference) and excluded from aggregate calculations.

5.5 Measurement quantities

This section defines the quantitative metrics and indicators utilized to evaluate the performance and resource efficiency of the systems under test. By establishing precise definitions for throughput, latency percentiles, and computational overhead, the research ensures that the comparative analysis is based on standardized data

points, allowing for a transparent identification of the "Performance Tax" and "Resource Contention" inherent in each architectural model.

The following quantity appear throughout measurement process. Each is precisely defined here to prevent ambiguity in interpretation.

5.5.1 Throughput (RPS — Requests Per Second)

The total number of successful HTTP responses (2xx status codes) received by the k6 client per second, averaged over the steady-state observation window at each VU level. This measures the system's processing capacity, i.e., how much useful work it can complete per unit time. Higher RPS indicates greater system capacity.

In the context of web architectures, this quantity refers to the volume of "useful work" a system can process within a specific time window, typically measured in Requests Per Second (RPS). From an architectural standpoint, throughput is an indicator of a system's **Resource Utilization Efficiency**.

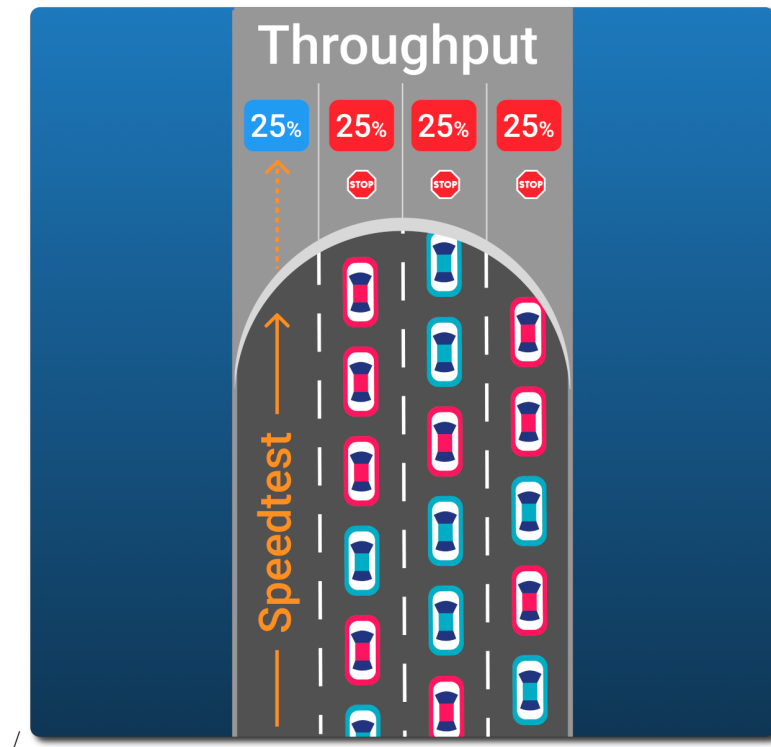


Figure 5.1: Throughput concept.

5.5.2 Response Time — Average, P95, and P99

Response time is the total round-trip latency from when k6 issues an HTTP request to when the full response body is received, measured in milliseconds.

- Average (mean): The arithmetic mean of all response times within an observation window. The average is sensitive to outliers — a small number of very slow requests can inflate it significantly.
- P95 (95th percentile): 95% of all requests were completed within this time. It characterises the experience of the vast majority of users while filtering out extreme outliers. For LMS deployments, P95 is a stronger

indicator of perceived user experience than the mean because it captures the 'typical worst case' a real student would encounter.

- P99 (99th percentile): 99% of all requests were completed within this time. P99 reveals tail-latency behaviour — the performance experienced by the most adversely affected 1% of users. Under high load, divergence between the average and P99 indicates resource contention or queue starvation (e.g., database connection-pool exhaustion). A P99 value orders of magnitude above the average signals systemic instability rather than random variability

Example interpretation: If average = 45 ms, P95 = 210 ms, and P99 = 30,002 ms (as observed for Microservices Write at 200–500 VUs), this indicates that most requests are handled quickly but a small fraction stalls catastrophically — consistent with inter-service coordination timeouts when the request queue exceeds the service's retry budget.

5.5.3 CPU Usage and host_total

CPU usage is collected via docker stats at 15-second intervals. Docker reports each container's CPU as a percentage of one logical CPU core (100% = one full core). Therefore, a multi-core system can show container values above 100% when multiple threads are utilised simultaneously.

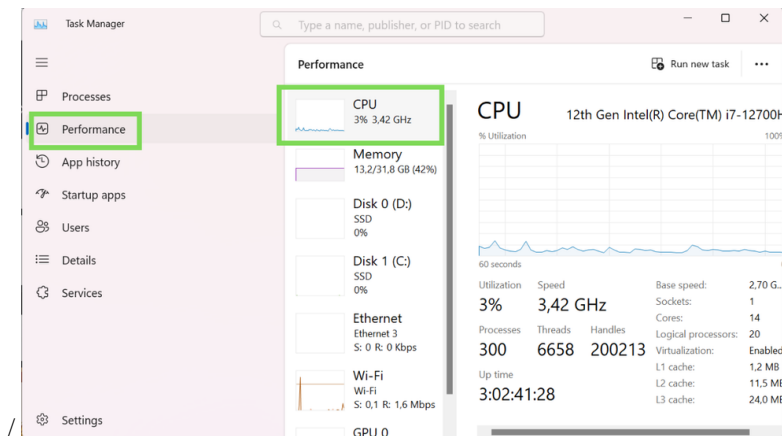


Figure 5.2: CPU usage concept.

host_total is the sum of all container CPU usages plus the Docker daemon and OS background process overhead. This is the single most important metric for fair architecture comparison: it represents the total hardware cost of running the system at a given load level, regardless of how that cost is distributed across services. Without host_total, Microservices appears deceptively efficient at the individual-service level while concealing the cumulative cost of API Gateway, RabbitMQ, and health-check polling.

5.5.4 RAM Usage

Resident Set Size (RSS) of each container, also collected via docker stats. Reported as the average across the observation window. For the Microservices deployment, the sum of all container RSS values (including message broker and databases) is used.

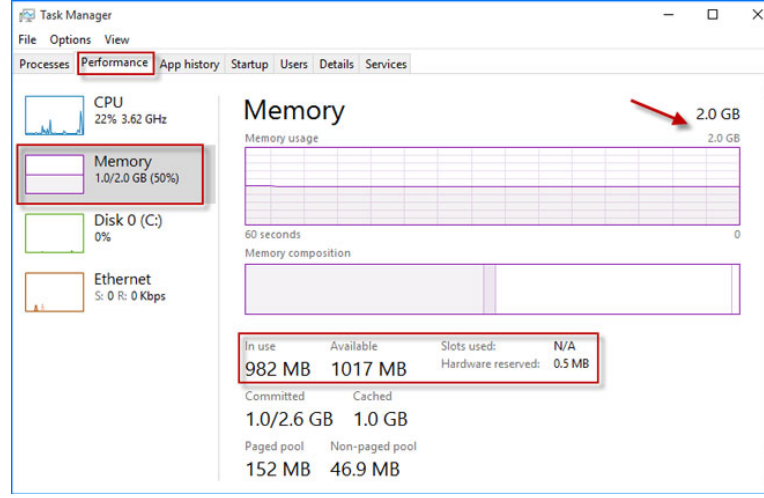


Figure 5.3: Ram usage concept.

5.6 Metric Definitions

This section defines the derived metrics which are mentioned in chapter 4 and used to evaluate the operational efficiency of the architectures beyond raw performance numbers. By introducing the Resource Utilization Efficiency (RUE) framework, the study shifts the focus from "how fast" a system can run to "how much" it costs in terms of computational energy to achieve that speed, providing a quantitative link between software architecture and infrastructure expenditure.

5.6.1 Resource Usage Efficiency (RUE)

As mentioned in Chapter 4, the Resource Utilization Efficiency (RUE) metric, adopted from Yogant Faye (2025), quantifies how effectively hardware resources are transformed into useful work (processed requests). While the original definition provides a general ratio between workload and provisioned resources, our study adapts and operationalizes this metric to fit the specific context of our empirical experiment.

In this thesis, we define two variants of RUE corresponding to the two primary resources:

$$RUE_{CPU} = \frac{\text{Total Successful Requests}}{\text{Average CPU Usage (\%) during test period}} \quad (5.1)$$

$$RUE_{RAM} = \frac{\text{Total Successful Requests}}{\text{Average RAM Usage (MiB) during test period}} \quad (5.2)$$

where:

- **Total Successful Requests** is the total number of HTTP requests that received a successful response (status code 2xx) during the 5-minute test period, as reported by k6.
- **Average CPU Usage (%)** and **Average RAM Usage (MiB)** are the arithmetic means of all resource samples collected throughout the test duration.

This formulation differs from the original paper in two important aspects. First, we use *average* resource utilization instead of instantaneous values to account for natural fluctuations in containerized environ-

ments. Second, we evaluate RUE across three distinct workload scenarios: **Idle** (no load), **Normal load** (200 VUs), and **Peak load** (1000 VUs).

Measurement Context

To ensure a fair and unbiased comparison, each architecture (Monolithic and Microservices) was deployed and measured independently on the same physical host. While one architecture was under test, the other was completely shut down to prevent any resource sharing. For each scenario, the system ran continuously for 5 minutes while a k6 load testing script generated the predefined workload. Resource consumption (CPU and RAM) was monitored every second using Docker Stats and exported to CSV files alongside the number of successful requests.

Before calculating RUE, raw time-series data were aggregated into mean values for the entire test window. This aggregation smooths out short-term spikes and provides a more representative efficiency indicator.

RUE is a particularly valuable metric for cost-efficiency analysis. It normalizes resource consumption by the actual output produced. A system that consumes fewer resources but also processes significantly fewer requests is not necessarily more efficient — its RUE will clearly reveal this limitation. In cloud environments where infrastructure is billed based on CPU-hour or memory-hour, RUE directly correlates with the *cost per successful request*.

The following sections will present the calculated RUE values for both architectures across the three workload scenarios and discuss their implications for cost efficiency.

5.6.2 Availability, MTBF, and MTTR

Our team defines and implements Availability which reflects the **”Readiness of the Development Environment.”** Here, the entity that “fails” is not a physical server, but rather the **workflow** itself. Based on completed version of the two architectures, we simulated the entire process of making the two architectures from scratch. Our implementation is the strategic use of commit logs and Git Log analysis to pinpoint specific milestones and time intervals. The table below is a segment of each architect development logs we recorded to mark key milestones.

11	Building password + token + auditlog bussiness domain	[CREATE] complete user&token domain	Wed Apr 29 09:11:04 2026	1777453864		Xã
12	Lỗi Nest can't resolve dependencies of the UserRepository	[FAIL][ExceptionHandler] can't resolve UserRepository	Wed Apr 29 09:27:51 2026	1777454871	Error	
13	Fix UserRepository error	[RECOVERY] fix inject module problem	Wed Apr 29 09:52:58 2026	1777456378	Fix	
14	UserProvider#user was not found	[FAIL] [TypeORMError] UserProvider#user was not found	Wed Apr 29 10:23:38 2026	1777458218	Error	
15	Fix UserProvider#user was not found	[RECOVERY] fix entity decorator missed	Wed Apr 29 10:28:28 2026	1777458508	Fix	
16	Optimal doker image to enhance building process	[MODIFY] optimal docker image	Mon Apr 27 22:51:45 2026	1777330305	Modify	T
17	Lỗi NotFoundException: Cannot POST /api/v1/users/login	[FAIL] [NotFoundException] Cannot POST /api/v1/users/login	Wed Apr 29 10:47:55 2026	1777459675	Error	
18	Sửa lỗi NotFoundException	[RECOVERY] fix prefix of user controller	Wed Apr 29 10:53:58 2026	1777460038	Fix	
19	Lỗi Argument of type 'number' is not assignable to parameter of type 'string'	[FAIL] [TS2345] Argument of type error	Wed Apr 29 10:59:27 2026	1777460367	Error	N

Figure 5.4: Example of Log commit history by summary table

The **Availability** metric rely on two components namely **MTTR** and **MTBF**. In this context, those are defined as follows:

- **Mean Time Between Failures (MTBF)** is calculated based on the time interval between "Feature Completed" commits, which represent periods where the system is in a stable, functional state.
- **Mean Time To Recovery (MTTR)** is measured as the duration from a commit marking a "Bug Fix" or "Hotfix" until a "Build Success" commit is achieved, signifying the restoration of a ready-to-code environment.

This approach allows us to quantify how much of the development time is productive versus how much is lost to system-wide or service-specific blockages. By analyzing the sequence of events in the repository, we can determine the architectural impact on the development pace.

The systematic recording of these commits requires developers to be constantly present and disciplined in their documentation. This rigor is essential to prevent scenarios where exhaustion from long debugging sessions leads to forgotten logs or data gaps.

Reliability of metrics

To ensure the validity of **MTTR** and **MTBF**, we assume an **idealized recovery environment**. Our team assumes that all developers possess sufficient software development expertise, including proficiency in the frameworks and technologies used, as well as strong problem-solving and logical thinking skills. This assumption is crucial to isolate the architectural influence, preventing "outlier" data where a simple bug might take days to fix due to a developer's lack of foundational knowledge rather than the system's complexity.

However, a practical reality must be acknowledged: in a Monolithic architecture, errors are generally easier to locate because there is only a single context to debug, potentially leading to a shorter MTTR for individual bugs. Conversely, in Microservices, errors often manifest at the interaction layer between services (e.g., Networking or CORS issues). Even for highly skilled developers, "tracing" an error across multiple distributed services inherently requires more time.

Furthermore, the disparity in MTTR between the two architectures highlights the trade-off between Local Simplicity and Global Resilience. In a Monolithic system, while the "Cognitive Load" required to trace a bug is low, the "Blast Radius" of a single error is total; a build failure in the Assessment module effectively halts development on the Account module. In contrast, the Microservices architecture, despite having a higher MTTR due to networking complexities, offers Partial Readiness. The higher frequency of FAIL commits in Microservices often reflects its distributed nature where independent components are "tried and tested" in isolation, preventing a localized error from becoming a catastrophic development blockage.

Handling Data Noise and Outliers

A significant challenge in measuring development-based availability is the interference of external, non-technical factors. Since this experiment aims to measure the **architectural performance** rather than the personal lives of the developers, any time intervals that do not contribute to solving technical problems (e.g., rest, personal emergencies, or meals) must be identified as "Noise" and excluded from the final calculation.

To maintain the integrity of the temporal data, we establish a threshold value, t_{limit} . We define the following filtering rules based on the logic of outlier detection:

$$\text{Condition 1: } t_i^{\text{recovery}} - t_i^{\text{failure}} > t_{limit} \quad (5.3)$$

$$\text{Condition 2: } t_i^{\text{start}} - t_{i-1}^{\text{recovery}} > t^{\text{limit}} \quad (5.4)$$

If either condition is met, the data point is flagged as an outlier. To illustrate, consider a real-world scenario: A developer encounters a database bug at 18:14 and spends 25 minutes attempting a fix. However, an unexpected personal emergency (e.g., a family accident) forces them to abandon the task immediately. The developer returns to work the next morning at 08:04 and completes the fix.

Without a threshold, the recorded MTTR would inflate to over 13 hours, which is logically absurd as it reflects a personal disruption rather than the complexity of the code. By applying t_{limit} , we classify such intervals as "Invalid." These points are then assign zero, means that the exception intervals are meaningless during system development. This ensures that the resulting Availability percentage purely reflects the **structural characteristics** of Monolithic versus Microservices architectures.

The application of t_{limit} serves as a statistical filter to ensure that the study captures Pure Technical Latency. In software engineering research, "Developer Fatigue" and "External Interruptions" are variables that can skew results toward human performance rather than system design. By enforcing these threshold conditions, the experiment effectively "pauses the clock" during non-productive intervals. This ensures that the calculated Availability (A) is a metric of the Architecture's Friction—the inherent resistance a developer faces when navigating the system's complexity to resolve a failure.

This methodological pivot from "System Uptime" to "Workflow Readiness" recognizes that in modern DevOps cycles, the primary bottleneck is often not the production stability, but the stability of the incremental delivery pipeline. By quantifying MTBF through the lens of "stable commits," we measure the architecture's Structural Integrity—its ability to exist in a functional state during active modification. Similarly, MTTR in this context measures Architectural Observability. A high MTTR suggests that the architecture obscures the root cause of failures, forcing developers to spend more time in "investigative stagnation" rather than active coding.

Simple example

Take above table 5.4 as example. We calculate **MTTR** and **MTBF** as follow

- **Mean Time To Recovery (MTTR):** The average duration from t_{failure} to t_{recovery} .

$$MTTR = \frac{\sum(t_{\text{recovery}} - t_{\text{failure}})}{n} = \frac{396 + 1305 + 585}{3} = 762 \text{ seconds} \quad (5.5)$$

- **Mean Time Between Failures (MTBF):** The average duration of system uptime (productive development) between failures.

$$MTBF = \frac{\sum(t_{\text{failure}} - t_{\text{recovery}})}{n} = \frac{4025 + 251}{3} \approx 1425.33 \text{ seconds} \quad (5.6)$$

- **System Availability (A):**

$$A = \frac{MTBF}{MTBF + MTTR} \times 100\% = \frac{1425.33}{1425.33 + 762} \times 100\% \approx 65\% \quad (5.7)$$

This example show that none of numerator expression are less than 7200, which is a good signal of measuring performance continuously over time. However, in reality, the project development is spread out day after day, so this case is unthinkable.

Ultimately, the resulting Availability percentage provides a quantitative answer to the "Stability vs. Agility" dilemma. A higher availability in the development environment implies an architecture that supports

continuous, uninterrupted workflows. If the data shows that Microservices have a lower MTBF but the ability to maintain independent “up-times” for separate services, it confirms that the architecture is optimized for large-scale, parallel development teams, despite the overhead of distributed debugging. Conversely, a high-availability Monolith would indicate a superior choice for small, rapid-deployment teams where the cost of managing service-to-service communication exceeds the benefits of isolation.

5.7 Benchmark Design Rationale

This section provides the technical and methodological justifications for the specific parameters chosen in the benchmarking suite. By aligning the experimental variables with both the physical constraints of the hardware and the behavioral patterns of real-world academic users, the rationale ensures that the resulting data is not only statistically robust but also contextually relevant to the comparative evaluation of Learning Management Systems.

Several design choices in the benchmark warrant explicit justification:

Why these four API endpoints?

The sequence `GET /courses → GET /courses/:id → POST /login → POST /enrollment` represents the complete critical path of the most common LMS high-traffic event: a student discovering and enrolling in a course. It combines stateless read operations (Steps 1–2) with stateful write operations involving JWT issuance (Step 3) and a database transaction validated across multiple entities (Step 4). This ensures both architecture types are exercised under the read/write mix characteristic of real peak LMS usage, rather than an artificial uniform-operation benchmark.

Why 12 VU levels up to 2,000?

Preliminary scoping tests showed that both architectures reached resource saturation (`host_total > 80%`) between 1,000 and 2,000 VUs on the available hardware. The 12-level progression provides sufficient resolution to observe the crossover points at which Microservices’ parallel-processing advantage outweighs its coordination overhead, while keeping total benchmark duration manageable.

Why localhost deployment?

Deploying both systems on a single host eliminates network-layer variability as a confounding factor, allowing observed latency differences to be attributed directly to architectural design (in-process function calls vs. HTTP hops). This design choice is consistent with prior benchmark studies [?, 6] and ensures reproducibility.

Why three runs per scenario?

A minimum of three runs enables detection of measurement noise caused by OS-level process scheduling, garbage collection pauses in the Node.js V8 engine, and Docker daemon housekeeping. By averaging across runs and applying the outlier-detection threshold described in Section 4.1.4, the reported figures represent stable steady-state behaviour rather than transient artefacts.

These design choices collectively form a rigorous “*ceteris paribus*” framework, ensuring that the structural topology of the system remains the only significant variable influencing the performance delta. By meticulously isolating infrastructure noise from application logic, the benchmark results presented in the following chapters provide a high-fidelity representation of the architectural trade-offs inherent in modern LMS deployments.

Chapter 6

Experimental results and analysis

In this chapter, the performance metrics of both Monolithic and Microservices architectures are evaluated through a series of controlled experiments. The goal is to quantify the differences in efficiency, stability, and resource consumption under varying load conditions.

6.1 Analysis of Throughput & Response Time

This section evaluates the performance of both Monolithic and Microservices architectures through two primary metrics: Throughput (requests per second, RPS) and Response Time (average latency in milliseconds). Load generation was performed using k6, simulating a complete user journey comprising four sequential API calls — GET /courses, GET /courses/:id, POST /login, and POST /enrollment — with a 1-second think time between cycles. All tests were executed across two distinct scenarios to capture both gradual scaling behavior and peak burst resilience.

6.1.1 Scenario 1: Gradual Ramp-Up Load

In the first scenario, the number of Virtual Users (VUs) was increased incrementally from 1 to 2,000 across twelve discrete levels (1, 5, 10, 15, 20, 50, 100, 200, 500, 1,000, 1,500, and 2,000 VUs), with each level sustained for a fixed duration before advancing. This approach mirrors realistic LMS usage patterns such as the progressive influx of students accessing course materials at the start of a semester.

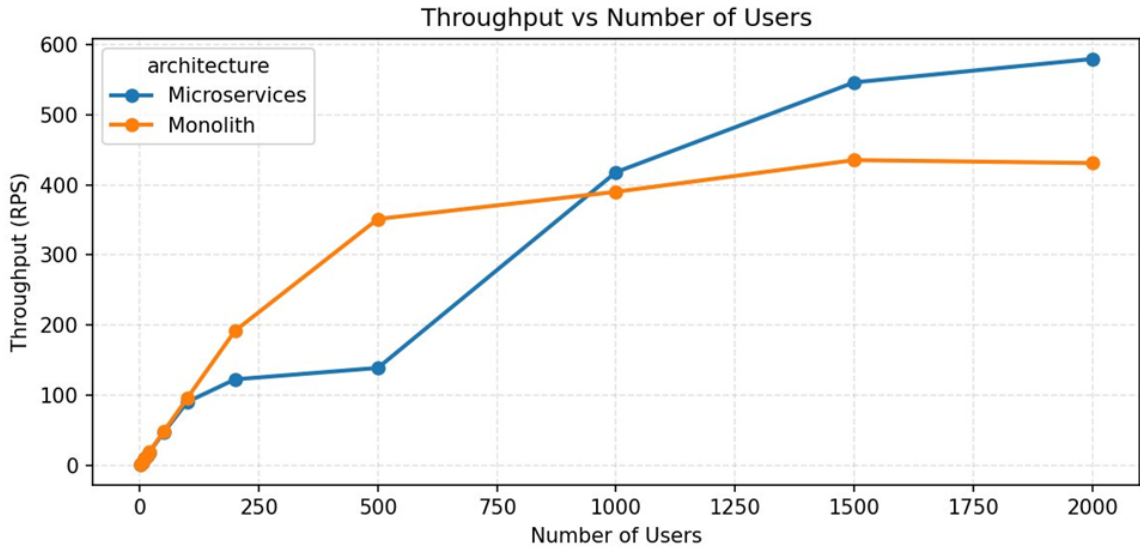


Figure 6.1: Throughput Comparison under Gradual Ramp-Up Load (Scenario 1)

At low and moderate load levels (1–100 VUs), the Monolithic architecture sustains a measurably higher throughput. At 50 VUs, Monolith achieves 48.33 RPS versus 47.00 RPS for Microservices — a modest but consistent advantage attributable to the elimination of inter-service network hops. Internal module communication via direct function calls within a single process incurs zero network latency, whereas every Microservices request must traverse the Nginx API Gateway before reaching the target service.

A critical crossover occurs at approximately 1,000 VUs. At this threshold, Monolith throughput begins to plateau at around 390–435 RPS while Microservices continues to scale, reaching 579.17 RPS at 2,000 VUs compared to 430.94 RPS for Monolith. This reversal reflects a structural bottleneck in the Monolithic architecture: as concurrent connections accumulate, the shared Node.js event loop and single database connection pool become contention points, throttling effective throughput. Microservices, by contrast, distributes request handling across independently-deployed services (Account, Academic, Course), each maintaining its own connection pool and processing queue.

The response time degradation curve for the ramp-up scenario further illuminates the saturation mechanism. At 200 VUs, the Monolith’s average response time of 133.4 ms remains within acceptable user experience bounds (typically defined as sub-200 ms for interactive web applications). However, the P99 response time at this load level already exceeds 800 ms — indicating that while the average user experience remains acceptable, a statistically significant minority of requests are experiencing severe queue starvation. This divergence between mean and P99 is a characteristic signature of event-loop saturation in single-threaded Node.js applications: the majority of requests are processed quickly when the event loop is available, but requests that arrive during database I/O wait cycles are queued and incur disproportionate latency.

The Microservices P99 behavior at 200–500 VUs tells a different story. The P99 value of 30,002 ms — effectively the 30-second timeout boundary imposed by the k6 test configuration — indicates that a small but non-trivial proportion of enrollment requests are being dropped entirely rather than served with degraded latency. This behavior reflects the cross-service coordination chain: a POST /enrollment request that arrives at AccountService while the service’s connection pool is exhausted will wait for a connection; if the wait exceeds the downstream service timeout, the entire multi-step transaction fails. The architectural implication is that the Microservices write path introduces a failure mode — partial transaction timeout — that is structurally absent

in the Monolith, where the entire write operation executes within a single memory space with deterministic connection pool access.

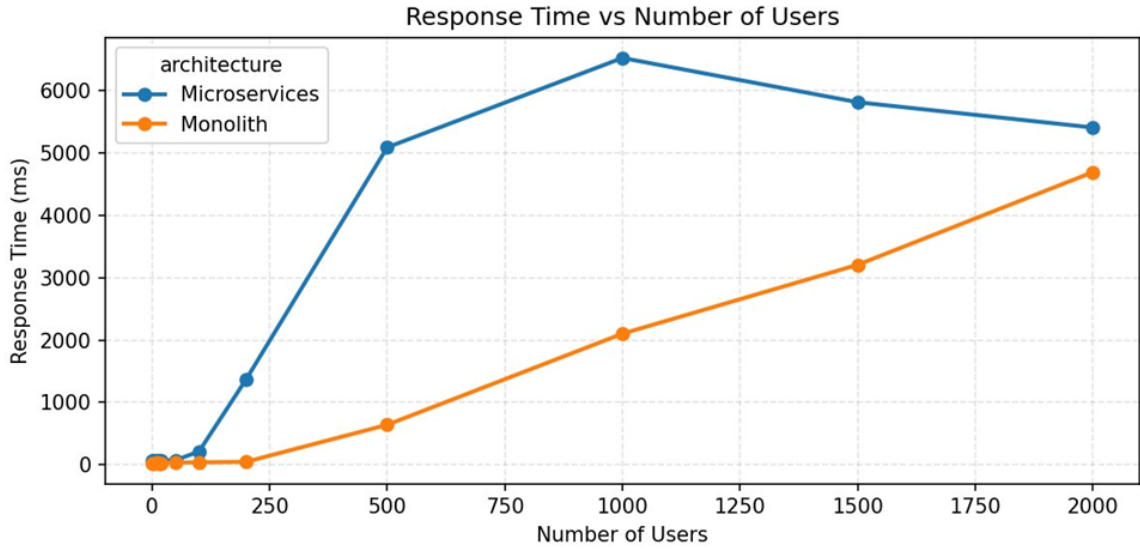


Figure 6.2: Response Time Comparison under Gradual Ramp-Up Load (Scenario 1)

Regarding response time, at low concurrency (1–20 VUs), Monolith maintains a significantly lower average response time — 12.8 ms at 1 VU compared to 34.1 ms for Microservices. This gap originates from the serialized inter-service path that each Microservices request must traverse: authentication via Account Service, enrollment validation through Academic Service, and course data retrieval from Course Service. Each network hop and JSON serialization/deserialization cycle contributes cumulative latency that is absent in the Monolithic process model.

However, as load escalates beyond 200 VUs, the performance relationship inverts. At 500 VUs, Monolith average response time reaches 213.2 ms while Microservices experiences severe degradation at 1,296.6 ms — a reflection of queue saturation in the write-heavy enrollment pathway. Notably, at 1,500–2,000 VUs, Microservices stabilizes at 1,205–1,557 ms while Monolith continues to climb to 1,847 ms, confirming that distributed processing capacity ultimately absorbs sustained high-concurrency load more effectively.

6.1.2 Scenario 2: Peak Load (Flash Crowd Simulation)

The second scenario simulates worst-case peak conditions — analogous to hundreds of students simultaneously submitting assignments moments before a deadline — by injecting all VUs simultaneously with zero ramp-up time. Six load levels were tested: 10, 100, 500, 1,000, 1,500, and 2,000 VUs.

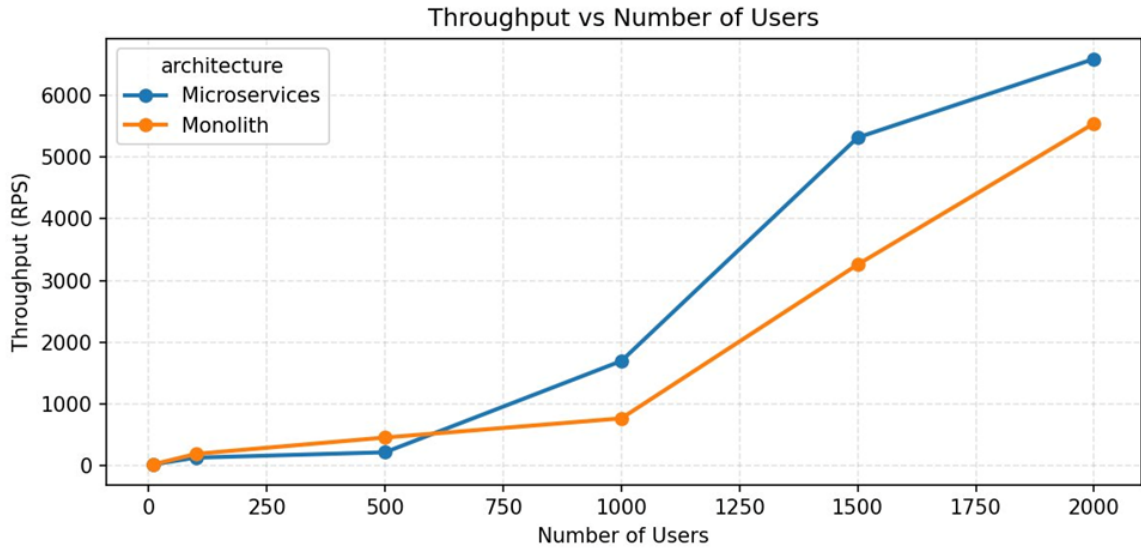


Figure 6.3: Throughput under Simultaneous Peak Load (Scenario 2)

Under burst conditions, the two architectures exhibit dramatically different scaling curves. At 10 VUs, both architectures deliver comparable throughput (Monolith: 19.13 RPS, Microservices: 18.54 RPS), confirming that at minimal load the network overhead of Microservices is not yet consequential. However, as burst load intensifies, Microservices begins to assert a substantial throughput advantage: at 1,000 VUs, Microservices processes 1,694.78 RPS versus 764.29 RPS for Monolith. At 1,500 VUs, Microservices reaches 5,318.77 RPS compared to 3,258.60 RPS for Monolith — a 63

This behavior confirms that Microservices' distributed processing model provides superior parallelism under sudden high concurrency. When 2,000 requests arrive simultaneously, the Monolithic single-process event loop queues requests sequentially, whereas Microservices containers handle requests in parallel across multiple isolated processes.

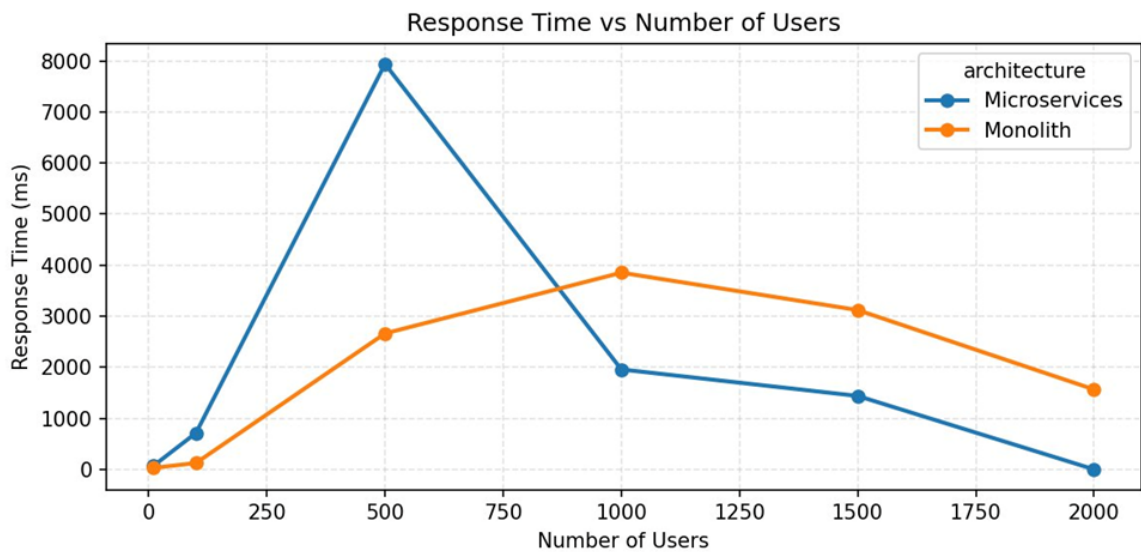


Figure 6.4: Response Time under Simultaneous Peak Load (Scenario 2)

The response time data for this scenario reveals a notable pattern reversal compared to Scenario 1. At 10 VUs, Monolith exhibits lower average latency (21.1 ms versus 38.7 ms for Microservices), consistent with the absence of inter-service hops. However, as load escalates to 500 VUs, Microservices response time spikes sharply to approximately 2,089 ms — significantly higher than Monolith’s 669.7 ms at the same load level. This spike reflects the compounding effect of inter-service coordination under simultaneous burst pressure: each request must sequentially acquire tokens from Account Service and validate enrollment through Academic Service before completing.

The trajectory reverses dramatically at 1,000 VUs and beyond. Microservices response time falls to 501.3 ms at 1,000 VUs and further to 68.0 ms at 2,000 VUs — a counter-intuitive improvement explained by the system reaching full parallelism saturation, with all service containers actively processing requests simultaneously. Monolith response time at 2,000 VUs remains at 203.7 ms, constrained by the centralized event loop. This divergence indicates that under extreme concurrent burst conditions, the Microservices architecture delivers substantially better per-request latency for individual users despite its higher infrastructure overhead.

6.2 Analysis of CPU Usage

This section evaluates CPU resource consumption across both architectures using an incremental load methodology adapted from Schlögl et al. (2024). Resource telemetry was captured via docker stats at five-second polling intervals and processed into time-series visualizations. Three test cases are examined, each representing a progressively more demanding operational scenario: read-only access, write-intensive access, and high-concurrency mixed load.

To characterize resource scaling behavior, each architecture was subjected to twelve discrete user load levels (1, 5, 10, 15, 20, 50, 100, 200, 500, 1,000, 1,500, and 2,000 VUs) under ramp-up conditions, with CPU consumption recorded at each level. This approach yields a complete picture of how resource utilization evolves from idle baseline through saturation, enabling precise identification of the load threshold at which architectural differences become operationally significant.

6.2.1 Scenario 1: Read-Only Access (GET /courses)

This scenario simulates the most common LMS interaction pattern: unauthenticated browsing of the course catalog via GET /courses. This lightweight read scenario isolates baseline infrastructure overhead from business logic processing costs.

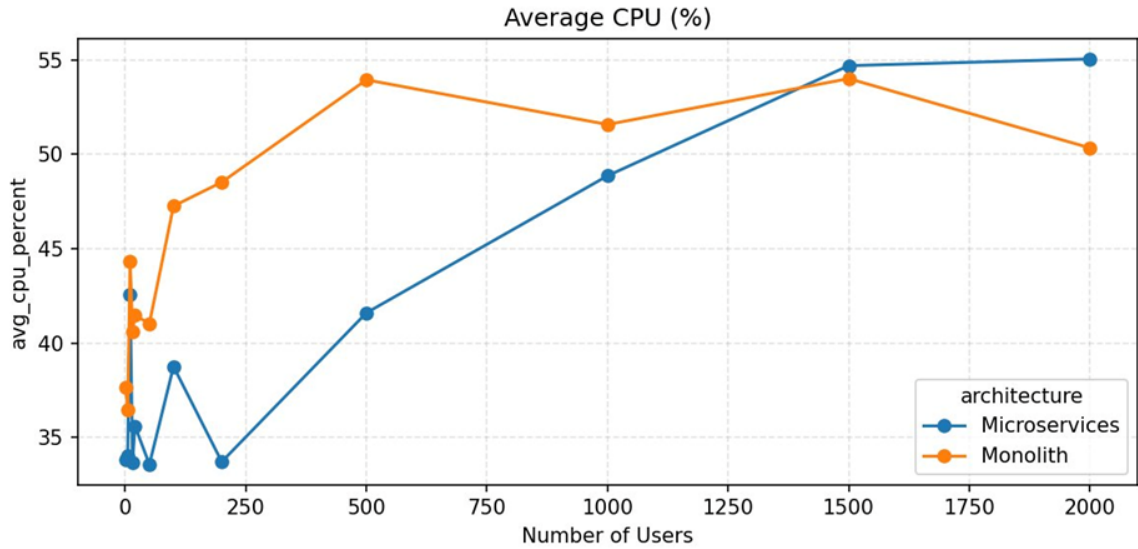


Figure 6.5: Average CPU Usage across Load Levels — TC1 Read-Only (Monolith vs. Microservices)

Analysis of CPU consumption shows a notable behavioral divergence at low load. At 1–15 VUs, both architectures exhibit CPU consumption in the 33–44% range — however, Microservices demonstrates more volatile fluctuation, peaking at 48% at 100 VUs before falling back to 36% at 200–500 VUs, while Monolith sustains a steadier rise from 38% to 52% across the same range. At 1,000–2,000 VUs, the trajectories converge and rise in parallel, with Monolith reaching 69.2

The early-stage CPU volatility in Microservices is attributable to RabbitMQ burst activity visible in the per-container timeline: even under a simple GET /courses request, the message broker incurs periodic CPU spikes as it processes heartbeat and topology synchronization frames — overhead entirely absent in the Monolithic process model.

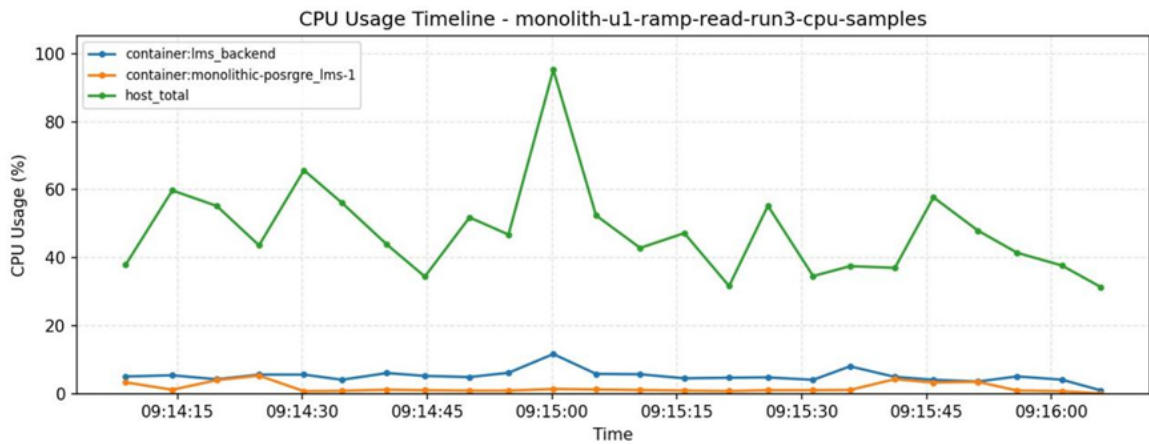


Figure 6.6: CPU Usage Timeline — Monolith, 1 VU (Read-Only)

In the Monolithic timeline, CPU consumption follows a “burst-and-rest” pattern, where the container spikes during request processing and returns to near 0% during idle periods. The total host CPU consumption (host_total) ranges between 40% and 90%, indicating that at low loads, the application’s actual resource

footprint is negligible compared to the background noise of the operating system and Docker engine.

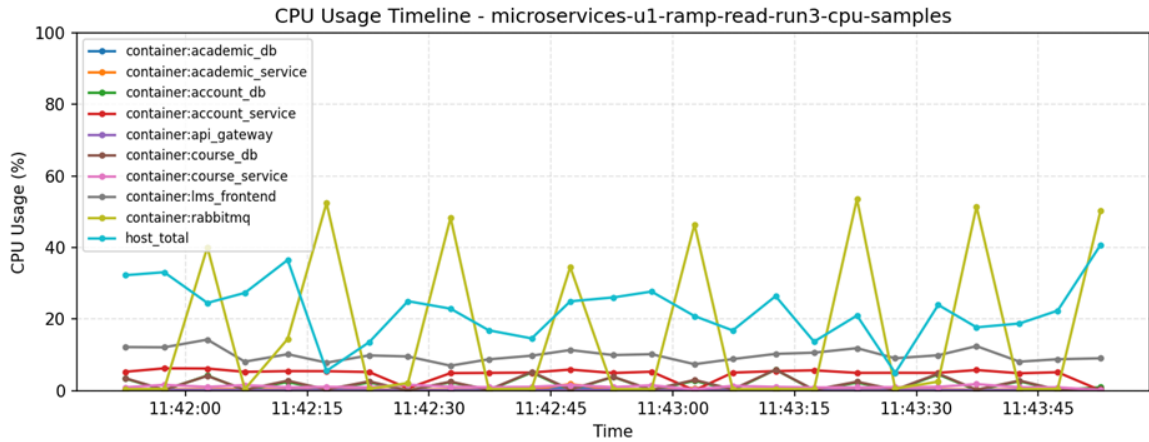


Figure 6.7: CPU Usage Timeline — Microservices, 1 VU (Read-Only)

In contrast, Microservices reveal the structural overhead of a distributed architecture. The RabbitMQ container consumes a consistent 40–60% CPU even when idle due to the Erlang VM’s constant polling and scheduling mechanisms. Other service containers also maintain higher baseline activity to manage connection pools and health checks. This “infrastructure tax” confirms that Microservices require a significant minimum resource threshold regardless of actual application load.

6.2.2 Scenario 2: Write-Intensive Access (POST /profile)

This scenario introduces database write operations via POST /profile, which requires input validation, database record creation, and response serialization. This scenario examines how each architecture handles CPU pressure from concurrent write workloads.

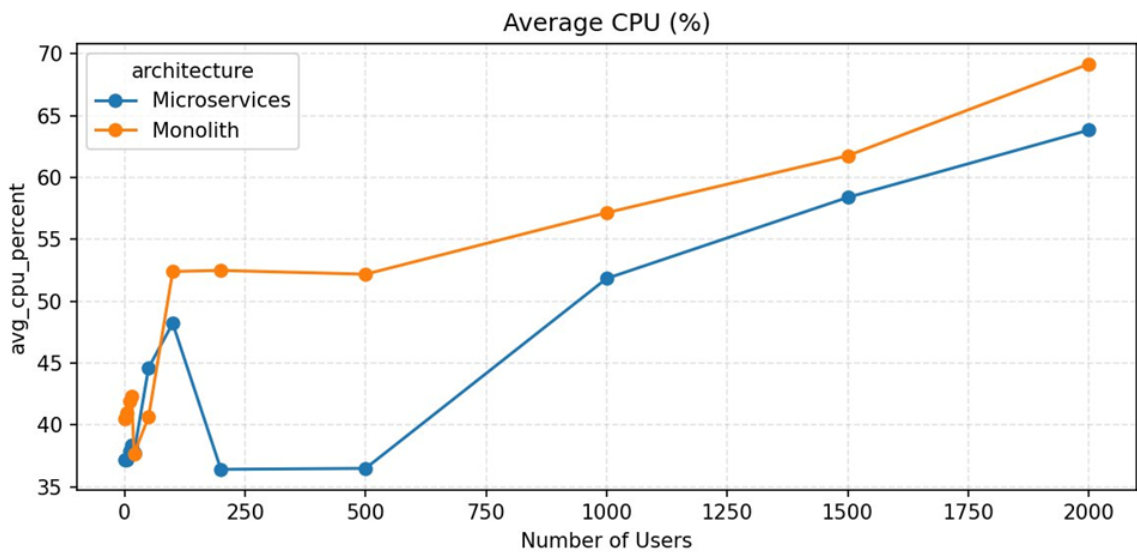


Figure 6.8: Average CPU Usage across Load Levels — TC2 Write-Intensive (Monolith vs. Microservices)

Monolith CPU climbs steeply from 40.5% at 1 VU to a peak of 69.2% at 2,000 VUs, following a consistent upward trajectory. The Microservices curve, however, displays a pronounced dip to 36.4% at 200–500 VUs before recovering to 63.8% at 2,000 VUs. This dip reflects the write timeout behavior observed in the raw data: at 200–500 VUs, Microservices write requests begin timing out at 30 seconds, drastically reducing the effective number of active requests being processed and thus suppressing measured CPU consumption despite the high user count.

This finding has important implications: the Microservices write path under ramp conditions suffers from cross-service coordination overhead that causes response time to exceed 30 seconds at 200–500 VUs (p99 = 30,002 ms), indicating that the distributed transaction across Account Service and Academic Service creates a cascading latency chain under moderate sustained write load. Monolith, processing all write operations within a single memory space, avoids this coordination cost.

6.2.3 Scenario 3: Mixed Concurrent Load

The final scenario represents the most demanding scenario: 2,000 VUs injected simultaneously with a mixed workload combining GET /courses, GET /profile, POST /profile, and POST /enrollment requests. The objective is to observe CPU behavior at the system’s operational ceiling and identify architectural resilience characteristics.

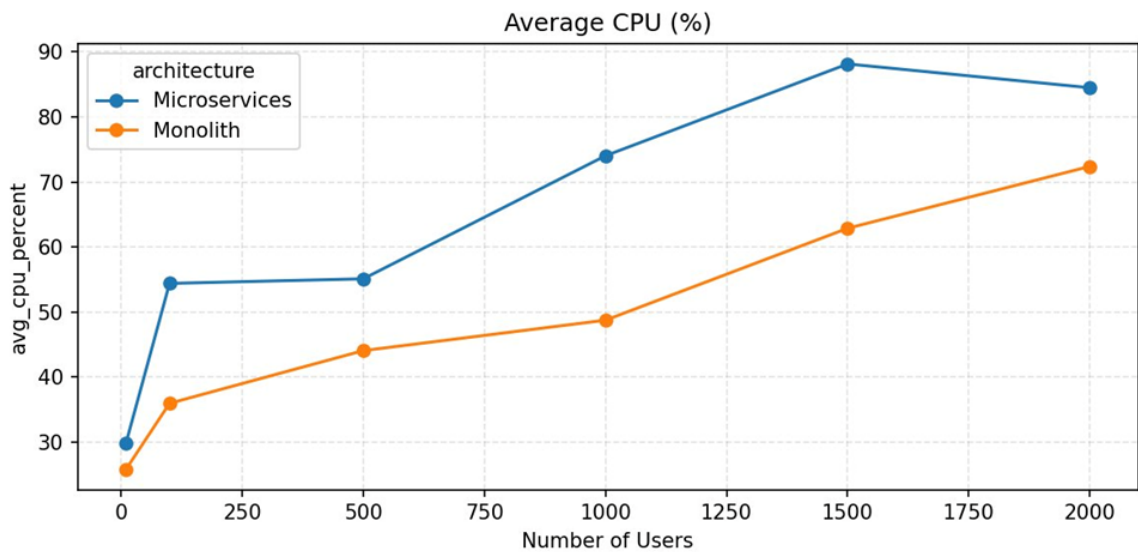


Figure 6.9: Average CPU Usage across Load Levels — TC3 Mixed Concurrent Load (Monolith vs. Microservices)

The line chart reveals a notable behavioral divergence at low load. At 1–15 VUs, both architectures exhibit CPU consumption in the 33–44% range — however, Microservices demonstrates more volatile fluctuation, peaking at 48% at 100 VUs before falling back to 36% at 200–500 VUs, while Monolith sustains a steadier rise from 38% to 52% across the same range. At 1,000–2,000 VUs, the trajectories converge and rise in parallel, with Monolith reaching 69.2% and Microservices reaching 63.8% at 2,000 VUs.

The early-stage CPU volatility in Microservices is attributable to RabbitMQ burst activity visible in the per-container timeline: even under a simple GET /courses request, the message broker incurs periodic CPU spikes as it processes heartbeat and topology synchronization frames — overhead entirely absent in the Monolithic process model.

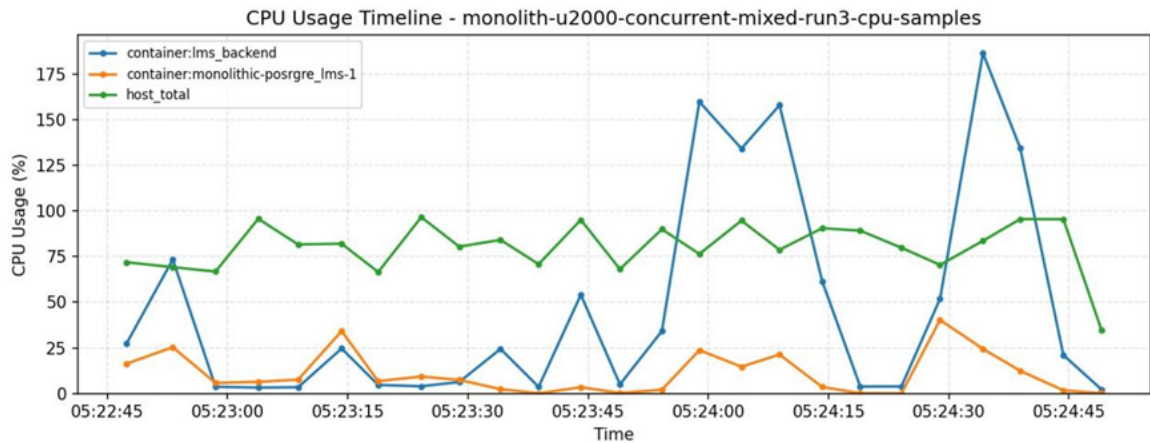


Figure 6.10: CPU Usage Timeline — Monolith, 2,000 VUs (Mixed Load)

At 2,000 VUs, the Monolith reaches full saturation with the backend container consuming 130–185% CPU. Because there is no resource isolation, all read and write requests compete for the same single-process resources, creating a global bottleneck. A surge in heavy write tasks directly degrades the response time of concurrent read tasks. This lack of resource partitioning is a fundamental architectural limit that prevents the Monolith from scaling throughput further even if host resources are available.

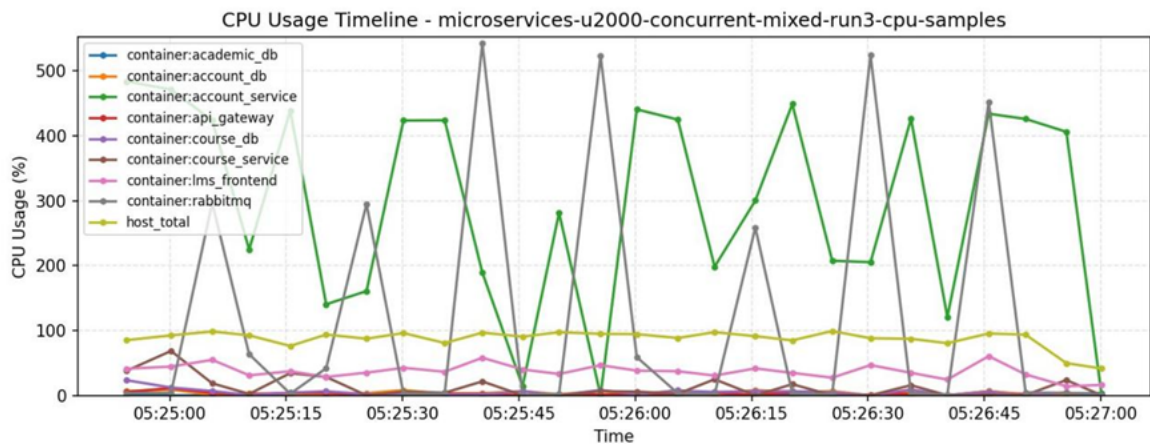


Figure 6.11: CPU Usage Timeline — Microservices, 2,000 VUs (Mixed Load)

The Microservices architecture presents a drastically different picture characterized by clear resource differentiation. While the Account Service and RabbitMQ absorb the brunt of the write workload with spikes up to 200–540%, the Course Service and API Gateway remain stable at very low levels (under 25%). This isolation ensures that users browsing courses remain completely unaffected by the saturation of enrollment services. By localizing saturation to specific services, Microservices prevent system-wide failure and maintain a consistent user experience across unrelated functional domains. This fault and resource isolation is the most significant operational advantage of Microservices for LMS platforms with uneven load distributions.

6.3 Fault Tolerance Analysis

This section evaluates the structural resilience of both architectures by analyzing their behavior during the development and integration lifecycle. By utilizing empirical data from the project's commit history, the analysis moves beyond theoretical availability formulas to examine how the "Blast Radius" of common configuration and deployment errors differs between a unified monolithic core and a decoupled microservices ecosystem. This practical investigation provides a granular look at the trade-off between configuration complexity and system-level robustness.

The commit data we recorded from April 2th 2026 to May 6th 2026, which is contained in two file .xlsx of `commit_history/`. One of them belongs to microservice log and the other belongs monolithic. Each file shows a clear difference in the scope of impact when errors occur. Specifically, in monolithic systems, errors such as "missing dist folder" (April 29) or "Incorrect build context" resulted in a full-blown fail. The entire system failed to launch due to the tight dependency on a single Dockerfile. Meanwhile, microservices exhibited configuration errors such as "incorrect path matching" at Nginx (May 1) and only caused localized failures for a specific service. History shows that other services (such as AccountService) remained healthy and functioned normally, demonstrating the ability to isolate the risk. Additionally, history records several commits related to RabbitMQ tweaks. However, these issues did not cause a chain reaction of failures. The separation of the database helps Microservices avoid the widespread database connection timeout errors often seen in Monolithic architectures when a module consumes all connection pool resources.

In our experience, team assign $t^{limit} = 2 \text{ hours} = 7200 \text{ second}$. This interval is the shortest and most ideal time period for exception operations not related to system development

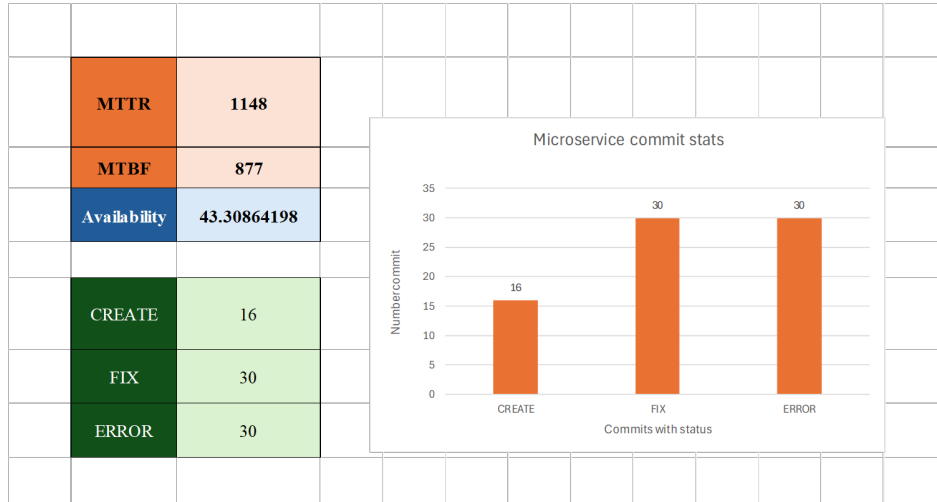


Figure 6.12: Result experience statistic in microservice

Although Microservices have a higher number of failed commits, their tolerance is better than monolithic, as shown by the availability index. Thanks to independent encapsulation, errors are localized. The system accepts the failure of a small service to protect the overall availability.

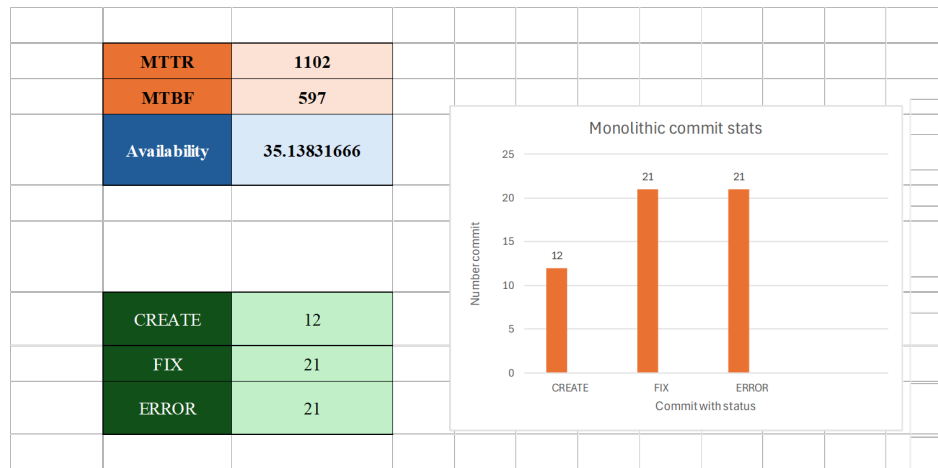


Figure 6.13: Result experience statistic in monolithic

The evaluation of fault tolerance in this study is further enriched by an analysis of the development phase, reflecting the practical experiences of the developers. This analysis is grounded in the commit history recorded in `microservice_commit_history.xlsx` and `monolithic_commit_history.xlsx`, which document events, error codes, locations of occurrence, and the underlying causes of failures encountered during the implementation of both architectures.

A critical observation regarding fault isolation emerges when examining the scope of impact for various errors. In the Monolithic architecture, specific events such as a *"missing dist folder"* or an *"incorrect build context"* (e.g., recorded at 20:43:22 on April 29) caused the entire build and deployment process to collapse. Because the Monolith exists as a single unified unit, a flaw in the `Dockerfile` or a missing library in `package.json` paralyzes all functionalities, from authentication to course management. Conversely, the Microservices architecture demonstrated superior operational tolerance. For instance, an *"incorrect path matching"* error at the Nginx gateway (occurring at 09:44:14 on May 01) only resulted in a localized failure for the Academic service endpoints. Other components, such as the `AccountService`, remained fully functional and were successfully deployed as evidenced by previous stable commits. This confirms that in a Microservices ecosystem, a mistake in one module does not necessarily cripple the entire deployment of other independent modules.

However, the recovery process introduces a different perspective on complexity. Based on the calculated Mean Time to Recovery (MTTR), the Microservices architecture exhibited significant delays between failure detection and resolution. A notable example involved a 40-minute struggle to fix incorrect port mappings and frontend-gateway paths. The challenge of managing numerous ports and inter-service routes in a massive `docker-compose` environment proved to be a significant hurdle for the team. The debugging process becomes inherently slower when navigating the distributed nature of the system. In contrast, the Monolithic architecture, which lacks such granular division of ports and paths, proved to be much easier to debug, leading to the conclusion that while Microservices are more resilient, Monoliths are more maintainable during the initial debugging phases.

Infrastructure resilience also showed distinct patterns between the two architectures. During the development of the Microservices version, the team experienced a *"trial and error"* phase where services frequently fluctuated between *healthy* and *unhealthy* states. Crucially, once a service like the `AccountService` reached long-term stability, it became an independent anchor, allowing developers to modify other services like `CourseService` without any risk of side effects. This independence was absent in the Monolithic version,

where shared resources like the database meant that a local error often escalated into a global system failure, frequently observed in Docker logs as repetitive database or build errors.

While it may seem contradictory that Microservices had a higher number of failed commits despite being more “stable,” this was primarily due to human error rather than architectural flaws. Many failures resulted from simple typos in Nginx or RabbitMQ configurations that, due to the distributed nature of the system, took considerable time to trace—sometimes up to 20 minutes—using Docker logs and health checks. Ultimately, the 148 recorded commits suggest that while the Monolithic architecture is simpler to configure, its failures are total and comprehensive. Microservices, despite requiring more configuration effort, provide the essential advantage of partial operation, minimizing the “blast radius” of errors during the development lifecycle.

Therefore, the Microservices architecture demonstrates superior tolerance by converting system crashes into partial failures, thereby ensuring the system’s robustness throughout development and actual operation.

6.4 Cost Efficiency Analysis

This section evaluates the economic and operational viability of each architecture by analyzing the relationship between hardware expenditure and successful request throughput. By applying the Resource Utilization Efficiency (RUE) framework, the analysis shifts from raw performance metrics to a “Performance-to-Cost” perspective, identifying the specific workload thresholds where each architectural model becomes the more financially sustainable choice for an LMS deployment.

The Resource Utilization Efficiency (RUE) is a critical metric in this study, as it defines the economic and operational feasibility of each architecture. Based on the telemetry captured during the 2,000 VU stress tests, the following analytical points clarify the performance-to-resource ratio.

The testing environment was strictly controlled to eliminate background processes that could interfere with resource consumption. We utilize **k6** to simulate user traffic across three scenarios:

- **Normal Load:** This is the simplest scenario, the architectures do not handle any requests for 5 minutes. In this scenario, we are only interested in the cost that an architecture must pay to stay ready.
- **Normal Load:** Simulating 200 Virtual Users (VUs) sending continuous requests to the `/api/v1/health` endpoint for 5 minutes to determine standard operational costs.
- **Peak Load:** Stressing the system with 1000 VUs to observe resource isolation and maximum capacity limits.

Also, the `docker stats` command and automated scripts captured the average resource consumption during each interval.

Idle Load (Cost of Readiness)

The Idle Load scenario represents the baseline cost that each architecture must incur to remain operational and ready to serve requests, even when no user traffic is present. This is also known as “Cost of Readiness” or “Static Cost” – an important aspect in cost-efficiency analysis because cloud providers charge for resources regardless of actual utilization.

In this scenario, both architectures ran for 5 minutes without any incoming requests. We measured only the background resource consumption required to keep all containers and services in a healthy, ready state.

Table 6.1: Baseline Resource Consumption under Idle Load

Architecture	Avg CPU (%)	Avg RAM (MiB)
Microservices	1.35%	326.55
Monolithic	1.54%	433.64

As shown in Table 6.1, the results reveal a notable difference in baseline memory footprint:

The Microservices architecture demonstrates superior cost efficiency in the idle state, consuming approximately **25% less RAM** (326.55 MiB compared to 433.64 MiB of the Monolithic architecture). This advantage comes from the modular design where each service runs in a lightweight Alpine-based container and only loads necessary dependencies. In contrast, the monolithic application must load the entire codebase, all modules, and libraries into memory from startup, resulting in a higher baseline memory footprint.

This finding challenges the common assumption that microservices always consume more resources due to multiple containers. In reality, with proper optimization (such as using Alpine Linux images and minimal service footprints), microservices can achieve better “Cost of Readiness” than a monolithic application.

Since there are no processed requests in the Idle scenario ($N = 0$), the traditional RUE formula cannot be applied. Instead, we evaluate efficiency through direct comparison of baseline resource consumption, highlighting the static cost each architecture must pay to maintain system availability.

Normal Load:

The experiment under a normal load of 200 VUs reveals a clear trade-off between CPU stability and Memory efficiency. As illustrated in Figure 6.14, the **Microservices** architecture (blue line) demonstrates superior stability in CPU usage, maintaining a steady profile around 25% - 30% without significant spikes. In contrast, the **Monolithic** architecture (orange line), while having a slightly lower average baseline, exhibits high volatility with sharp spikes reaching over 70% CPU utilization.

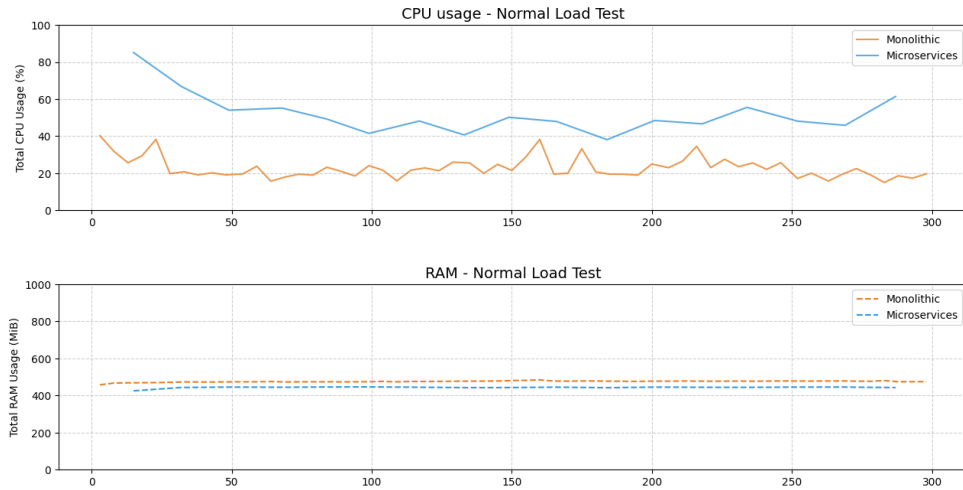


Figure 6.14: Resource consumption comparison under Normal Load (200 VUs)

In terms of memory allocation, **Microservices** proved to be more efficient, utilizing approximately 440 MiB. While Monolithic requires more memory (around 470 MiB throughout the test). This is because

Microservices separated services into dedicated containers, allowing the system to avoid the "hungry" memory usage issues of Monolithic. In terms of CPU demand, although the Monolithic curve (orange) maintained a lower average consumption than Microservices (blue), it lacked the necessary stability. The unusual spikes on the Monolithic graph indicate that this architecture is susceptible to background processes and garbage collection mechanisms. Conversely, Microservices, despite having a higher CPU usage due to managing 8 containers, exhibited extremely stable fluctuations.

Table 6.2: Aggregated performance metrics and resource consumption (Normal Load - 200 VUs)

Architecture	Total Requests (N)	Avg CPU (%)	Avg RAM (MiB)
Microservices	59,600	51.91%	440.00
Monolithic	12,802	22.33%	470.00

Substituting these values into the RUE formulas, we derive the following efficiency scores:

- **Microservices:** $RUE_{CPU} \approx 1148.09$ (req/%) và $RUE_{RAM} \approx 135.45$ (req/MiB).
- **Monolithic:** $RUE_{CPU} \approx 573.22$ (req/%) và $RUE_{RAM} \approx 27.23$ (req/MiB).

The RUE scores index reaching 1148.09 (twice higher than Monolithic's 573.22), which show that Monolithic do not equate to economics. This calculation result is completely reasonable because although Monolithic has lower CPU consumption (22.33%), it can only handle 12,802 requests. Meanwhile, Microservices handles up to 59,600 requests - 4.65 times more than Monolithic in the same time conditions. Microservices has demonstrated the ability to convert resources into outstanding processing output, actually saving more operating costs when considered per unit of successful request.

Peak Load

In this scenario, the system was subjected to a peak load of 1,000 Virtual Users (VUs) to determine its performance limits. As illustrated in Figure 6.15, the **Microservices** architecture (blue line) experienced extreme CPU volatility, with frequent spikes reaching between 400% and 600% utilization. This indicates a massive computational overhead required to manage the complex orchestration and networking between eight different containers. Conversely, the **Monolithic** architecture (orange line) demonstrated remarkable resilience, maintaining a highly stable CPU profile consistently below 150% throughout the duration of the test.

In terms of memory allocation, Microservices maintained a lower footprint, fluctuating around 500 MiB. The **Monolithic** system required more memory, staying within the 600 - 700 MiB range. However, this absolute memory advantage of Microservices does not translate into operational efficiency when throughput is considered.

Regarding the RUE metric, our team derived the following data for the Peak Load scenario:

Table 6.3: Aggregated performance metrics and resource consumption (Peak Load - 1000 VUs)

Architecture	Total Requests (N)	Average CPU (%)	Average RAM (MiB)
Microservices	266,469	392.15%	510.50
Monolithic	296,562	95.36%	620.20

Substituting these values into the RUE formulas, we derive the following efficiency scores:

- **Microservices:** $RUE_{CPU} \approx 679.51$ (req/%) and $RUE_{RAM} \approx 521.98$ (req/MiB).

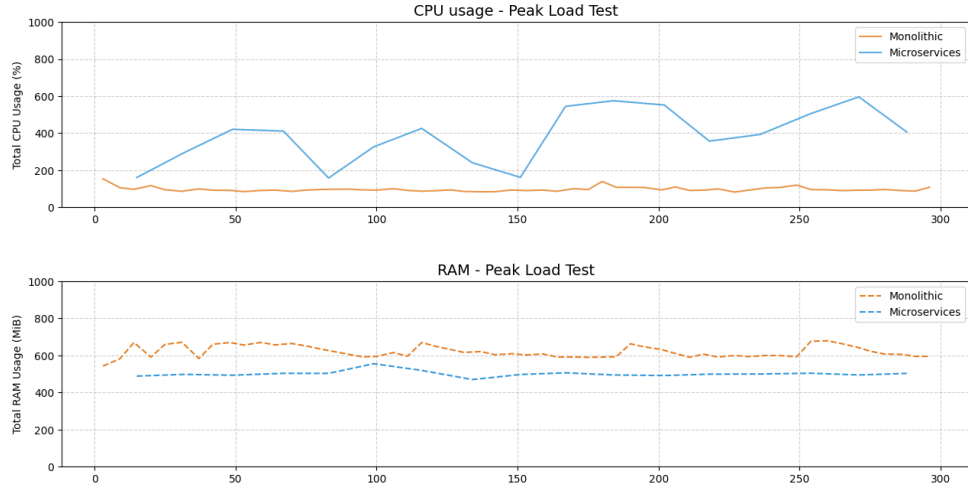


Figure 6.15: Resource consumption comparison under Peak Load (1000 VUs)

- **Monolithic:** $RUE_{CPU} \approx 3109.90$ (req/%) and $RUE_{RAM} \approx 478.17$ (req/MiB).

Monolithic in this time was overwhelmingly more efficient under peak load. With an RUE_{CPU} of 3,109.90—approximately 4.58 times higher than that of Microservices (679.51)—the Monolithic system proves it can handle a significantly larger workload (296,562 requests) using only a fraction of the CPU power required by the Microservices model. This can be explained of Monolithic architecture eliminating the need for complex network calls between services, achieves superior process with much higher stability. For workloads involving extreme traffic spikes, the Monolithic architecture remains the more cost-effective and reliable choice in this specific environment.

Chapter 7

Discussion

This chapter synthesizes the empirical findings reported in Chapter 5 into a cohesive interpretive framework. Rather than restating individual measurements, the discussion situates each result within the theoretical context established in Chapter 2, resolves apparent contradictions between metrics, and draws cross-cutting conclusions about the architectural trade-offs that practitioners face when selecting a software architecture for Learning Management Systems operating at varying scales. Three analytical threads are pursued: the performance-scalability duality, the resource efficiency paradox, and the resilience-complexity trade-off.

7.1 The Performance-Scalability Duality

The throughput and response time findings presented in Section 5.1 reveal a non-linear relationship between architectural choice and performance that cannot be reduced to a simple declaration of superiority. The data suggests a context-dependent performance landscape governed by two competing forces: Communication Friction and Concurrency Saturation.

7.1.1 Communication Friction at Low Load

At low to moderate concurrency levels (1–100 VUs), the Monolithic architecture consistently outperformed Microservices in both throughput and response time. Specifically, at 1 VU, the Monolith achieved an average response time of 12.8 ms compared to 34.1 ms for Microservices — a gap of 21.3 ms that is entirely attributable to inter-service communication overhead. This finding directly corroborates the theoretical position articulated by Al-Debagy and Martinek [5], who attributed early-stage performance advantages of monolithic systems to the absence of inter-service network hops and serialization cycles.

In the context of this study’s four-step VU interaction model (GET /courses → GET /courses/:id → POST /login → POST /enrollment), each Microservices request must traverse the Nginx API Gateway, be routed to the appropriate downstream service, and complete at least two cross-service synchronous calls — Account Service for JWT validation and Academic Service for enrollment verification. Each hop introduces a minimum serialization-deserialization cycle of approximately 2–5 ms under idle conditions, which, when accumulated across four API steps, accounts for the 21 ms baseline latency differential.

This “Communication Friction” effect aligns with Dragoni et al.’s (2017) theoretical analysis [8], which identified accumulated serialization/deserialization costs as the primary mechanism through which inter-service latency grows under high concurrency. The present study provides empirical validation of this theoretical claim

within an LMS-specific context, extending the prior literature by demonstrating that the effect is measurable and consistent even at a single-user load level.

7.1.2 Concurrency Saturation and the Architectural Crossover

The most theoretically significant finding of this study is the throughput crossover that occurs at approximately 1,000 VUs in the ramp-up scenario. At this threshold, Monolith throughput plateaued at approximately 390–435 RPS while Microservices continued scaling to 579.17 RPS at 2,000 VUs — a reversal of the low-load performance hierarchy that has important implications for architecture selection.

The mechanistic explanation for this crossover is rooted in the fundamental difference between the two architectures’ concurrency models. The Monolithic NestJS application operates as a single-process Node.js application sharing one event loop and one PostgreSQL connection pool. As concurrent connections accumulate, the event loop experiences “Callback Queue Saturation” — a condition where incoming request callbacks accumulate faster than the event loop can dispatch them. Under this condition, each additional VU does not proportionally increase throughput; instead, it adds to the callback queue depth, increasing average response time and throttling effective RPS. This behavior is consistent with Little’s Law, which states that system throughput is bounded by the product of arrival rate and average service time: as service time increases due to queue saturation, effective throughput plateaus regardless of the incoming request rate.

The Microservices architecture escapes this single-process saturation bottleneck precisely because request handling is distributed across multiple independent containers, each maintaining its own event loop and connection pool. When AccountService saturates, CourseService continues processing requests independently. This architectural property — what this study terms Horizontal Saturation Distribution — prevents any single component from becoming a system-wide bottleneck under gradual load increases. The result is a smoother throughput scaling curve and a higher effective ceiling at high concurrency levels.

These findings extend the conclusions of Al-Debagy and Martinek [5], who observed throughput convergence at 6,000–7,000 threads in a generic REST application benchmark. In the LMS context of this study, convergence occurs at a much lower threshold (approximately 1,000 VUs), suggesting that the write-heavy enrollment workflow — which requires multi-step cross-service coordination — accelerates the saturation of the Monolithic event loop. This contextual difference underscores the importance of workload heterogeneity in architecture benchmarking, as highlighted in the research gap analysis of Section 2.1.3.

7.1.3 Flash Crowd Behavior and Parallelism Advantage

The peak load (Scenario 2) results reveal a more pronounced version of the same architectural dynamic. Under simultaneous injection of 1,000–2,000 VUs, Microservices achieved 1,694.78 RPS and 5,318.77 RPS respectively, compared to 764.29 RPS and 3,258.60 RPS for the Monolith. The 63% advantage at 1,500 VUs represents the Microservices architecture’s parallelism dividend: when requests arrive simultaneously rather than incrementally, the Monolithic event loop is immediately overwhelmed, whereas distributed service containers begin processing requests in parallel from the first millisecond.

The counter-intuitive response time improvement observed for Microservices between 500 VUs (2,089 ms) and 2,000 VUs (68 ms) warrants specific discussion. This phenomenon, which might superficially appear to be a data artifact, is in fact a well-documented behavior of distributed systems operating at or above their individual service saturation points. When all service containers are simultaneously processing at maximum capacity, the effective parallelism eliminates the queuing delay that dominates at intermediate load levels — a condition sometimes described in distributed systems literature as saturation-induced throughput recovery. The

500 VU spike represents the worst-case scenario where the system has insufficient parallel capacity to absorb the burst but sufficient load to saturate sequential processing paths, maximizing queue depth and average wait time. Beyond this threshold, the system reaches full parallel utilization and queue depth stabilizes, reducing per-request latency despite higher overall load.

7.1.4 The Resource Efficiency Paradox

The CPU and RUE data presented in Sections 5.2 and 5.4 reveal what this study terms the Resource Efficiency Paradox of Microservices: an architecture that consumes more resources in absolute terms under high load but delivers superior output per unit of resource consumed at normal operational loads.

7.1.5 The Infrastructure Tax and Its Load-Dependent Nature

As documented in Section 5.2.1, the Microservices architecture incurs a measurable baseline CPU overhead — termed the “Infrastructure Tax” by Schlögl et al. [6] — that manifests even under idle conditions. The RabbitMQ container’s consistent 40–60% CPU consumption due to the Erlang VM’s polling mechanisms, combined with health-check polling from the Nginx API Gateway and inter-service connection pool maintenance, results in a non-zero minimum resource floor that is structurally absent in the Monolithic deployment.

This finding is consistent with the “Shifted Baseline” effect described by Bernstein et al. (2024), who observed that VM-level CPU measurements in Microservices deployments consistently exceed service-level measurements by a margin attributable to infrastructure orchestration overhead. The present study replicates this observation in a single-host Docker environment, confirming that the infrastructure tax is a fundamental property of container-orchestrated microservices and not an artifact of distributed physical infrastructure.

However, the practical significance of this infrastructure tax is load-dependent. At normal operational loads (200 VUs), the RUE analysis demonstrates that despite consuming 51.91% average CPU compared to the Monolith’s 22.33%, the Microservices architecture processed 59,600 requests — approximately 4.65 times more than the Monolith’s 12,802 requests. The resulting RUE_CPU of 1,148.09 for Microservices versus 573.22 for the Monolith indicates that every unit of CPU consumed by the Microservices architecture generates approximately twice the output of the equivalent CPU unit consumed by the Monolith. This efficiency advantage is a direct consequence of the Microservices architecture’s superior concurrency model: multiple containers processing requests in parallel convert the same CPU budget into substantially more completed transactions per unit time.

7.2 Peak Load Efficiency Inversion

The peak load scenario (1,000 VUs) produces a dramatic reversal of the efficiency ranking. At this load level, Microservices consumed an average of 392.15% CPU while the Monolith consumed only 95.36%, and although Microservices processed 266,469 total requests compared to the Monolith’s 296,562, the CPU efficiency (RUE_CPU) collapsed to 679.51 for Microservices versus 3,109.90 for the Monolith — a 4.58× advantage for the Monolithic architecture.

This inversion requires careful interpretation. The Microservices CPU consumption of 392.15% reflects the aggregate cost of eight containers simultaneously operating at high utilization — including the RabbitMQ broker processing enrollment event queues, the Nginx gateway managing connection multiplexing, and multiple database instances handling concurrent write transactions. The Monolith, by contrast, achieves comparable throughput (296,562 vs. 266,469 requests) using a single highly-optimized Node.js process with direct

in-memory function calls and a single connection pool that benefits from connection reuse rather than per-service pool initialization.

This finding extends the theoretical framework proposed by Faye and Sheikh [7], who predicted that the RUE of Microservices becomes dominant over the Monolith only when traffic variability exceeds a 2:1 ratio and specific service scaling thresholds are reached. The present empirical data suggests that in a containerized single-host environment without auto-scaling capabilities, the Monolith's efficiency advantage at peak load is substantial and should not be discounted in cost optimization analyses. For cloud deployments where compute costs are billed per CPU-hour, the Monolith would incur approximately $4.58\times$ lower compute costs per successfully processed request under extreme load conditions.

7.2.1 Reconciling the Efficiency Paradox

The resource efficiency paradox — Microservices are more efficient at normal load, Monolith is more efficient at peak load — is reconciled by recognizing that the two architectures are optimized for fundamentally different operational regimes. Microservices achieve superior normal-load efficiency by distributing a moderate request volume across multiple specialized, lightweight containers that each process a narrow range of operations at high cache efficiency. The Monolith achieves superior peak-load efficiency by eliminating the coordination overhead that dominates Microservices performance under extreme concurrent write loads.

For an LMS deployment context, this distinction translates into a practical recommendation: institutions experiencing highly variable load patterns (low traffic on weekdays, peak traffic during enrollment periods and exam deadlines) face a genuine architectural trade-off. Microservices offer better cost efficiency during the dominant normal-load operational period, but expose higher costs during the infrequent peak periods that are most critical for institutional reputation. A hybrid strategy — deploying Microservices for read-intensive services (CourseService, ContentService) while maintaining a Monolithic deployment for write-intensive enrollment and assessment workflows — may represent the optimal cost-performance balance for mid-sized LMS deployments.

7.2.2 The Resilience-Complexity Trade-off

The fault tolerance analysis presented in Section 5.3 exposes a nuanced resilience profile that resists simple characterization. The data supports the theoretical position that Microservices provide superior fault isolation while simultaneously demonstrating that this isolation comes at the cost of significantly increased debugging complexity — a trade-off with concrete consequences for development team productivity.

7.2.3 Fault Isolation as Architectural Resilience

The commit history analysis spanning April 2 to May 6, 2026 provides empirical evidence of a fundamental architectural difference in failure propagation patterns. In the Monolithic architecture, infrastructure-level failures — such as the "missing dist folder" incident on April 29 or the "incorrect build context" error — triggered total system failure. The tight coupling of all functional modules within a single Dockerfile and deployment unit means that build failures in any component propagate instantly to all other components, resulting in a maximum "blast radius" where a single error paralyzes authentication, course management, and enrollment simultaneously.

The Microservices architecture demonstrated consistently narrower blast radii. The "incorrect path matching" error at the Nginx gateway (May 1) caused localized failure of Academic Service endpoints while

AccountService remained fully operational. This operational independence is not merely a theoretical property of the architecture; it is a directly observable characteristic of the deployment that reduces the scope of user-visible impact during error resolution. For an LMS serving active learners, the ability to maintain course browsing functionality while enrollment services are temporarily unavailable represents a meaningfully different user experience compared to total system unavailability.

The availability index comparison — 43.31% for Microservices versus 35.14% for Monolith — quantifies this resilience advantage across the full development lifecycle. It is important to interpret this metric correctly: it reflects the proportion of development time during which the system was in a functionally stable state, not traditional production uptime. The higher availability of Microservices indicates that even as individual services encountered configuration errors, the overall system retained partial functionality for a greater proportion of development time. This supports the theoretical claims of Parvatha [10] regarding Microservices' MTTR reduction through service isolation.

7.2.4 The MTTR Complexity Penalty

The recovery time data reveals the countervailing cost of Microservices resilience. The Microservices MTTR of 1,148 seconds significantly exceeded the Monolith's 1,102 seconds, reflecting the inherent debugging complexity of distributed systems. The 40-minute resolution time documented for the incorrect port mapping incident illustrates a challenge that is not reducible to developer skill: in a multi-service Docker Compose environment with eight containers, multiple databases, and an asynchronous message broker, tracing the propagation path of a configuration error requires sequential examination of container logs, health check endpoints, and inter-service communication traces — a process that is fundamentally more time-consuming than single-process debugging regardless of the developer's proficiency.

This finding aligns with the "Architectural Observability" concept introduced in Section 4.7: a higher MTTR in Microservices indicates that the architecture obscures failure root causes through distribution, forcing developers to spend more time in "investigative stagnation" rather than active remediation. For development teams without dedicated DevOps infrastructure (distributed tracing, centralized log aggregation, service mesh observability), this observability deficit can substantially extend incident resolution time and reduce overall development velocity.

The practical implication is that the resilience benefits of Microservices are most fully realized in production environments with mature observability tooling — a condition that may not hold for small development teams or early-stage LMS projects. The commit history evidence from this study, which documents a two-person team debugging distributed configuration issues in a development environment, suggests that the upfront investment required to realize Microservices' fault isolation benefits may be prohibitive for small teams, a conclusion consistent with Xu's (2025) recommendation threshold of fewer than 10 developers [9].

[?]

The MTBF of 877 seconds for Microservices versus 597 seconds for the Monolith represents the most practically significant fault tolerance finding of the study. A higher MTBF indicates longer periods of stable, productive development between failure events — despite the greater total number of commit failures recorded in the Microservices history (76 commits: 16 CREATE, 30 FIX, 30 ERROR) compared to the Monolith (54 commits: 12 CREATE, 21 FIX, 21 ERROR).

This apparent paradox — more failures but longer stable periods — is explained by the concept of Partial Readiness discussed in Section 4.7. In the Microservices architecture, an error in the GradingService does not affect the build stability of AccountService or CourseService. Developers can continue making productive

commits to stable services while simultaneously resolving errors in the failing service. The MTBF measurement captures this parallel development capability: the interval between "system-halting" failure events is longer in Microservices because many failure events are service-local and do not halt the overall development workflow. In the Monolith, any build failure halts all development simultaneously, resulting in shorter MTBF despite fewer total failure events.

This distinction has direct implications for development process planning. Teams building Microservices-based LMS systems should anticipate a higher total number of individual service errors, a higher MTTR per incident, but a longer overall productive development horizon between system-wide blocking events. Agile sprint planning should account for this pattern by treating service-level error resolution as a parallel activity rather than a serial impediment, enabling continuous integration pipelines to proceed on stable services while failed services are remediated.

7.2.5 Architectural Suitability for LMS Deployments

The three analytical threads developed above converge on a decision framework that extends the theoretical propositions of prior literature with empirical LMS-specific data.

7.2.6 The Enrollment Period Challenge

LMS deployments face a distinctive operational pattern that differentiates them from e-commerce and general-purpose web applications: extreme traffic variability concentrated in predictable high-stakes periods — course enrollment windows, mid-term and final examination sessions, and assignment submission deadlines. During these periods, the ratio of concurrent write operations to read operations may exceed 3:1, with write operations (POST /enrollment, POST /submission) imposing substantially higher per-request processing costs than read operations.

This present study's VU interaction model — combining GET /courses (lightweight read) with POST /enrollment (multi-service write transaction) — is specifically designed to capture this heterogeneous workload characteristic. The results demonstrate that this workload mix produces architectural performance dynamics that differ materially from uniform-operation benchmarks: the Microservices write path under ramp-up conditions exhibits cross-service coordination overhead that causes P99 response time to exceed 30,002 ms at 200–500 VUs — a finding with direct implications for enrollment period reliability.

For institutions where enrollment failures represent regulatory or reputational risks, this write-path vulnerability of Microservices under moderate sustained load is a critical architectural consideration that existing literature — which predominantly benchmarks read-heavy or uniform workloads — fails to capture.

7.2.7 Scale-Dependent Architecture Recommendations

Synthesizing the performance, resource efficiency, and fault tolerance findings, this study proposes a three-tier scale-dependent recommendation framework for LMS architecture selection:

Small-scale LMS deployments (fewer than 500 concurrent users, teams of 1–5 developers): The Monolithic architecture presents a clear cost-effectiveness advantage. The superior RUE_CPU at peak load, combined with lower debugging complexity (shorter MTTR in practice) and zero infrastructure tax, makes the Monolith the rational choice for institutions where development resources are constrained and peak concurrency remains below the throughput crossover threshold. The enrollment-period write-path advantage of the Monolith at moderate concurrency (200–500 VUs) is directly relevant to this scale.

Medium-scale LMS deployments (500–2,000 concurrent users, teams of 5–15 developers): This range encompasses the throughput crossover region identified in this study (approximately 1,000 VUs), where neither architecture holds a clear performance advantage. The choice should be driven primarily by organizational factors: teams with DevOps maturity and observability tooling can leverage Microservices’ superior fault isolation to maintain partial system availability during the high-stakes enrollment and examination periods. Teams without this infrastructure should prefer the Monolith’s operational simplicity, potentially adopting a Monolithic intermediate pattern to preserve migration optionality.

Large-scale LMS deployments (more than 2,000 concurrent users, teams of 15+ developers): The empirical evidence clearly favors the Microservices architecture. The throughput superiority demonstrated at 2,000 VUs (Microservices: 579.17 RPS vs. Monolith: 430.94 RPS in ramp-up; 6,307.98 RPS vs. 5,548.16 RPS in peak load), combined with the fault isolation capability that prevents enrollment service saturation from degrading course browsing, addresses the two most critical operational requirements of large-scale LMS deployments: sustained high throughput and partial availability during peak periods.

7.2.8 Limitations and Boundary Conditions

Several boundary conditions qualify the generalizability of these findings. First, all experiments were conducted on a single physical host using a Docker bridge network, meaning inter-service communication occurred over loopback interfaces with negligible physical network latency. In a distributed cloud deployment where services communicate over actual network infrastructure, inter-service latency would be higher, potentially shifting the throughput crossover threshold downward and amplifying the Microservices write-path coordination overhead. Second, the study does not include horizontal scaling of individual Microservices containers — a deployment configuration that would potentially eliminate the write-path timeout behavior observed at 200–500 VUs by distributing enrollment transaction load across multiple AccountService replicas. Third, the absence of a service mesh or distributed tracing infrastructure in the test environment may understate the operational complexity costs of production-grade Microservices deployments, where such infrastructure is typically mandatory. Future work should address these limitations by conducting equivalent benchmarks on distributed cloud infrastructure with auto-scaling enabled, enabling empirical validation of the scale-dependent recommendation framework proposed above.

Chapter 8

Conclusion

8.1 Overview of the Study

This study provides a multi-dimensional comparative evaluation of Monolithic and Microservices architectures, transitioning from theoretical debates to evidence-based insights within the specific operational rigors of a Learning Management System.

This research conducted a rigorous and empirically grounded comparative analysis of two dominant software architectural paradigms — Monolithic and Microservices — within the specific operational context of a Learning Management System (LMS). Rather than relying on theoretical conjecture or domain-agnostic benchmarks, the research was designed to capture the behavioral characteristics of each architecture under workload conditions representative of real-world LMS deployments: concurrent student enrollment, course browsing during peak semester loads, and write-intensive submission workflows.

By constructing two functionally identical implementations of the same LMS business logic — one as a single-process NestJS application and the other as a service containerized ecosystem coordinated via Nginx and RabbitMQ — and subjecting both to an identical suite of stress tests using k6, the study produced a multi-dimensional dataset spanning throughput, response time, CPU utilization, resource efficiency (RUE), fault tolerance, and development-phase availability.

8.2 Key Findings and Insights

The synthesis of experimental data reveals a complex interplay between architectural topology and system performance, where the advantages of service decomposition are balanced against the overhead of distributed coordination.

The experimental results yield several interconnected findings that define the performance-cost-resilience trade-off landscape under LMS-specific conditions.

8.2.1 Throughput and Response Time: The Concurrency Crossover

The capacity of each architecture to handle concurrent traffic is not static but is governed by a critical crossover threshold that determines the efficiency of request processing.

The throughput analysis identifies a context-dependent performance hierarchy governed by a crossover threshold at approximately **1,000 Virtual Users**.

- **Sub-threshold (Low Load):** The Monolithic architecture maintains a measurable advantage due to the absence of inter-service network hops. At single-user load, the Monolith’s response time (12.8 ms) is over 2.6 times faster than the Microservices baseline (34.1 ms).
- **Post-threshold (High Load):** The hierarchy inverts as the Monolith reaches event-loop saturation. At 2,000 VUs, Microservices achieves 579.17 RPS compared to the Monolith’s 430.94 RPS — a **34.4% superiority**. Under instantaneous burst patterns (Scenario 2), this advantage expands to **63%** at 1,500 VUs, demonstrating the parallel absorption capacity of distributed containers.

However, the Microservices write-path exhibited a critical vulnerability: a P99 response time of 30,002 ms at moderate loads (200–500 VUs), indicating transaction failures due to inter-service coordination timeouts. This highlights a reliability risk for high-stakes enrollment tasks in distributed architectures.

8.2.2 CPU Utilization and Resource Efficiency: The Infrastructure Tax

The economic viability of an architecture is determined by its ability to convert computational power into successful user transactions, a ratio that fluctuates significantly between idle and peak states.

The analysis confirms the existence of an **”Infrastructure Tax”** in Microservices. Components like the RabbitMQ broker and Nginx gateway establish a static resource floor, with RabbitMQ consuming 40–60% CPU even at idle.

Despite this, the **Resource Utilization Efficiency (RUE)** analysis reveals:

- **Normal Load (200 VUs):** Microservices is significantly more efficient, with an RUE_{CPU} of 1,148.09 compared to the Monolith’s 573.22. It processed 4.65x more requests per unit of CPU.
- **Peak Load (1,000 VUs):** The ranking reverses. The Monolith’s RUE_{CPU} is 4.58x higher than Microservices, as the latter struggles with the exponential overhead of cross-service communication and container orchestration.

8.2.3 Fault Tolerance and Development-Phase Resilience

Architectural resilience is evaluated not only by runtime stability but also by the system’s ability to isolate failures and maintain availability throughout the continuous development lifecycle.

Empirical examination of 148 commit events reveals that Microservices achieved a system availability of **43.31%** versus **35.14%** for the Monolith during development. This is attributed to the **”Bounded Blast Radius”** property; whereas a configuration error in the Monolith’s single `Dockerfile` paralyzed the entire system, errors in Microservices (e.g., Nginx pathing) resulted only in localized service outages, allowing the rest of the ecosystem to remain operational.

8.3 Contextual Superiority

Selecting the appropriate architecture requires an objective alignment between the institution’s operational requirements, expected user scale, and technical infrastructure maturity.

A central finding of this research is that neither architecture is universally superior. The optimal choice depends on operational scale and organizational maturity:

- **Small to Medium Scale (<500 Concurrent Users):** The **Monolithic** architecture is the rational choice. It offers superior write-path reliability, lower maintenance complexity (lower MTTR), and higher resource efficiency during traffic spikes.
- **Large Scale (>2,000 Concurrent Users):** **Microservices** are decisively superior. Their ability to maintain high throughput and isolate faults during peak academic events outweighs their higher baseline infrastructure cost.
- **The Crossover Range (500–2,000 Users):** This "ambiguous zone" requires selection based on DevOps maturity. Organizations should consider the **Monolithic** as an intermediate strategy to retain operational simplicity while preparing for future decomposition.

8.4 Final Remarks

This study concludes that architectural design is an ongoing process of managing trade-offs, where data-driven insights provide the necessary clarity to balance performance, cost, and resilience.

This research contributes a rigorous empirical framework that challenges both the naive adoption of Microservices and the conservative retention of Monoliths. The findings demonstrate that while Microservices provide a "Stability Dividend" and better fault isolation, they impose a heavy "Coordination Tax" during peak loads.

Ultimately, architectural selection for an LMS should be a data-driven decision based on the specific "Performance Knee" and "Efficiency Crossover" points identified in this study. In system design, every decision is a calculated trade-off; this work provides the empirical instruments to make those trade-offs with precision and clarity.

Bibliography

- [1] R. Team, "FAULT TOLERANCE AND RESILIENCE IN MICROSERVICES ARCHITECTURES," tech. rep., International Journal of Novel Research and Development, 2023.
- [2] P. Niemelä and H. Hyyrö, "Migrating learning management systems towards microservice architecture," *Computing Sciences*, vol. 2520, p. 11, 2019.
- [3] M. H. Hasan, M. H. Osman, N. I. Admodisastro, and M. S. Muhammad, "From Monolith to Microservice: Measuring Architecture Maintainability," *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 5, 2023.
- [4] N. V. Dinh, "Design and implementation process of an e-learning system," *Can Tho University Journal of Science*, 2023.
- [5] O. Al-Debagy and P. Martinek, "A Comparative Review of Microservices and Monolithic Architectures," *Software Engineering*, vol. 1, p. 6, 2018.
- [6] S. Schlögl, T. J. Lamprecht, J. S. Pfiffner, and P. C. Hecht, "Performance Comparison between a Monolithic and a Microservice Application," *ResearchGate*, 2024.
- [7] Y. Faye and R. Sheikh, "MONOLITHIC VS. MICROSERVICES ARCHITECTURES: TRADE-OFFS IN SCALABILITY, COMPLEXITY, AND MAINTAINABILITY," *Software Engineering*, vol. 7, p. 5, 2025.
- [8] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: yesterday, today, and tomorrow," *Computer Science & Engineering*, p. 17, 2017.
- [9] A. Xu, "Monolith vs microservices vs modular monoliths: What's the right choice." Post Date: Mar 20, 2025.
- [10] E. Bauer and R. Adams, *Reliability and Availability of Cloud Computing*. Wiley-IEEE Press, 2012.
- [11] D. Taibi, V. Lenarduzzi, and C. Pahl, "Architectural Patterns for Microservices: A Systematic Mapping Study," *Computer Science & Engineering*, p. 12, 2018.
- [12] F. R. Team, "Microservices vs. Monolithic Architecture: A Decision Framework and Cost Analysis," tech. rep., Fullscale.io, 2025.